Audit Completo e Piano di Stabilizzazione — Mini Cozy Room

Data: 21 Marzo 2026 **Versione Progetto**: Godot 4.5 | GDScript | GL Compatibility **Autore**: Renan Augusto Macena **Ambito**: Analisi completa di 26 script, 9 scene, 5 file dati, 5 test, 3 workflow CI

Team di Progetto

Figura 1 — Assegnazione compiti per membro del team

Renan Augusto Macena — Senior Architect & Project Supervisor

Ruolo: Architettura generale del progetto, sviluppo delle parti piu' complesse del backend, integrazione del lavoro di tutti i membri del team, e responsabile del delivery finale del progetto.

Capitoli di riferimento per il proprio lavoro:

- **Sezione 6.3** — SaveManager: correggere race condition auto-save, backup senza error checking, inventario non salvato su SQLite (problemi C1, C2, C3)
- **Sezione 6.5** — AudioManager: bounds check tracce vuote, memory leak ambience, crossfade tween safety (problemi A4, A5)
- **Sezione 6.6** — SupabaseClient: token auth plaintext, HTTP pool illimitato, timeout mancanti (problemi A10, A11)
- **Sezione 11, Fase 4** — Allineamento architetturale: eliminare coupling diretto tra singleton, introdurre nuovi segnali `settings_updated`, `music_state_updated`, `save_to_database_requested` (violazioni AR1-AR11)
- **Sezione 11, Fase 1** — Integrità dati: coordinare la correzione di tutti i problemi critici C1-C7
- **Sezione 11, Fase 5** — Supervisione della copertura test e verifica finale pre-delivery

---

Cristian Marino — CI/CD & Documentation Lead

Ruolo: Gestione completa delle pipeline di Continuous Integration (lint, test, security scan, build), configurazione dei workflow GitHub Actions, e redazione di tutta la documentazione tecnica del progetto.

Capitoli di riferimento per il proprio lavoro:

- **Sezione 9.3** — CI/CD: aggiungere linting dei file test con gdformat, migliorare parsing regex nello schema database-ci.yml
- **Sezione 6.7** — Logger: flush sincrono che blocca il game thread, log persi se file non disponibile, session ID con possibili collisioni (problemi A12, A13)
- **Sezione 6.8** — PerformanceManager: posizione finestra non persistita prima dello shutdown (problema A14)
- **Sezione 11, Fase 5** — Copertura test: configurare i nuovi test nel workflow CI, verificare che i 6 nuovi file test vengano eseguiti nella pipeline
- **Sezione 14** — Mantenere aggiornati i riferimenti e la documentazione tecnica del progetto

---

Mohamed & Giovanni — Game Assets, Core Logic & Design Lead

Ruolo: Creazione e gestione di tutti gli asset grafici e sonori del gioco, implementazione delle logiche interne del videogioco (decorazioni, personaggi, interazioni, gameplay), e design delle meccaniche di gioco.

Capitoli di riferimento per il proprio lavoro:

- **Sezione 8, characters.json** — Correggere typo percorso sprite `sxt` → `sx`, completare o rimuovere personaggio `male_black_shirt` incompleto (problemi C6, C7)
- **Sezione 7.7** — room_base.gd: aggiungere `_exit_tree()`, correggere race condition swap personaggio (problema A3)
- **Sezione 7.8** — decoration_system.gd: correggere rimozione duplicati item_id, aggiungere `_exit_tree()` (problema A15)
- **Sezione 7.9** — character_controller.gd: aggiungere null check su `_anim`, validare nomi animazione prima di `play()`
- **Sezione 7.11** — window_background.gd: correggere mismatch dimensione array layers/factors (problema C5)
- **Sezione 7.1-7.5** — Tutti i pannelli UI: aggiungere `_exit_tree()` con disconnessione segnali, correggere memory leak drag preview e FileDialog (problemi A1, A2, A6, A7)
- **Sezione 7.6** — drop_zone.gd: correggere cast Texture2D unsafe (problema A16)
- **Sezione 8, tracks.json** — Espandere catalogo musicale con tracce lo-fi e ambience

---

Elia Zoccatelli — Database Support

Ruolo: Assistenza sullo sviluppo e la manutenzione del layer database (SQLite locale, schema Supabase, migrazione dati, query e persistenza).

Capitoli di riferimento per il proprio lavoro:

- **Sezione 6.4** — local_database.gd: ridisegnare tabella `characters` (rimuovere account_id come PK), ristrutturare tabella `inventario` (spostare coins/capacita in accounts), aggiungere foreign key item_id (problemi C3, C4)
- **Sezione 6.4** — local_database.gd: propagare errori apertura database, aggiungere seed data per tabelle vuote, creare funzione `delete_inventory_item()` (problemi A17, A18)
- **Sezione 8, supabase_migration.sql** — Allineare schema PostgreSQL con le modifiche SQLite
- **Sezione 11, Fase 1.4** — Implementare le istruzioni dettagliate per la correzione dello schema (istruzioni C3 e C4 nella Sezione 12)
- **Sezione 11, Fase 3** — Validazione struttura dati: aggiungere `_validate_save_data()` in SaveManager, safety su version comparison (problema A8)

Indice

1. Introduzione e Scopo del Documento
2. Come Leggere Questo Documento
3. Glossario dei Concetti Fondamentali
4. Panoramica del Progetto
5. Metodologia di Audit
6. Risultati — Autoload Singleton
7. Risultati — Script UI, Room e Menu
8. Risultati — Scene e Dati
9. Risultati — Test e CI/CD
10. Classificazione dei Problemi
11. Piano di Stabilizzazione
12. Istruzioni Dettagliate per Correzione
13. Verifica e Testing
14. Riferimenti e Risorse
15. Riepilogo Statistico

1. Introduzione e Scopo del Documento

Cos'e' un Audit del Software?

Immaginate di portare la vostra auto dal meccanico per un controllo completo prima di un lungo viaggio. Il meccanico non si limita a controllare se l'auto si accende: controlla i freni, l'olio, i pneumatici, le luci, il motore, la cintura di sicurezza. L'obiettivo non e' dire "l'auto e' rotta", ma piuttosto "ecco cosa funziona bene, ecco cosa potrebbe dare problemi, ed ecco cosa va sistemato prima di partire".

Un **audit del software** e' esattamente la stessa cosa, ma applicata al codice di un programma. Si analizza ogni singolo file, ogni funzione, ogni connessione tra i vari componenti del progetto. Si cercano:

- **Bug** (errori nel codice che causano comportamenti inaspettati)
- **Rischi di sicurezza** (porte aperte attraverso cui qualcuno potrebbe fare danni)
- **Problemi di memoria** (il programma consuma sempre piu' risorse senza mai liberarle)
- **Problemi di architettura** (il codice e' organizzato in modo che modificare una cosa ne rompe un'altra)
- **Mancanze nei test** (non c'e' un modo automatico per verificare che le cose funzionino)

Perche' Facciamo Questo Audit?

Il progetto **Mini Cozy Room** ha una base solida e molte cose fatte bene. Tuttavia, come ogni progetto software in fase di sviluppo, ha accumulato problemi tecnici che, se non risolti, potrebbero causare crash, perdita di dati degli utenti e difficolta' nel manutenere il codice in futuro.

Questo documento serve a tre scopi:

1. **Mappare lo stato attuale**: sapere esattamente dove siamo, cosa funziona e cosa no.
2. **Dare priorita' alle correzioni**: non tutti i problemi hanno la stessa urgenza. Alcuni causano perdita di dati (urgentissimo), altri sono solo questioni di stile (possono aspettare).
3. **Fornire istruzioni concrete**: per ogni problema trovato, questo documento spiega esattamente come correggerlo, con codice di esempio e spiegazioni passo-passo.

Cosa Non e' Questo Documento

Questo documento **non** e' una critica al lavoro fatto. Scrivere software e' un processo iterativo: si costruisce, si testa, si migliora. Ogni progetto software al mondo ha bug. L'importante e' trovarli e correggerli prima che raggiungano gli utenti finali.

2. Come Leggere Questo Documento

Struttura delle Sezioni

Il documento e' organizzato in modo progressivo:

- **Sezioni 1-3**: Contesto e glossario. Leggete queste per prime se siete nuovi a Godot o allo sviluppo software.
- **Sezione 4**: Panoramica del progetto. Vi aiuta a capire come e' strutturato Mini Cozy Room.
- **Sezione 5**: Come abbiamo condotto l'analisi.
- **Sezioni 6-9**: I risultati dell'audit, divisi per area del progetto.
- **Sezione 10**: Tabella riassuntiva di tutti i problemi, per avere una vista d'insieme.
- **Sezione 11**: Il piano per correggere tutto, diviso in 5 fasi.
- **Sezione 12**: Istruzioni dettagliate, passo per passo, per le correzioni piu' importanti.
- **Sezione 13**: Come verificare che le correzioni funzionino.
- **Sezione 14**: Risorse per approfondire.

Livelli di Severita'

Ogni problema trovato ha un livello di gravita'. Pensate a questi livelli come ai colori di un semaforo, ma con piu' sfumature:

Severita'	Analogia	Significato
CRITICO	Fuoco in cucina: bisogna spegnerlo ADESSO	Il gioco perde dati dell'utente, crasha in modo irrecuperabile, oppure ha una falla di sicurezza grave. Questi problemi devono essere corretti prima di qualsiasi rilascio.
ALTO	Freni dell'auto che grattano: si puo' guidare, ma e' pericoloso	Il gioco ha memory leak (usa sempre piu' memoria), race condition (due operazioni si pestano i piedi), oppure funzionalita' rotte. Va corretto al piu' presto.
MEDIO	Spia del motore accesa: il motore funziona ma qualcosa non va	Manca validazione degli input, errori che passano inosservati (silent failure), o codice poco manutenibile. Va corretto prima del rilascio finale.
BASSO	Graffio sulla carrozzeria: esteticamente sgradevole ma non pericoloso	Questioni di naming, best practice non seguite, ottimizzazioni minori. Si possono correggere quando c'e' tempo.
ARCHITETTURALE	Fondamenta della casa leggermente storte: la casa sta in piedi, ma aggiungere piani sara' difficile	Il modo in cui i componenti comunicano tra loro crea dipendenze che rendono il codice fragile e difficile da modificare.

Quale Sezione Leggere in Base al Proprio Lavoro

- **Lavorate sull'interfaccia utente (pannelli, menu)?** Concentratevi sulla Sezione 7.
- **Lavorate sui dati (salvataggio, database)?** Concentratevi sulle Sezioni 6.3, 6.4 e 8.
- **Lavorate sull'audio?** Concentratevi sulla Sezione 6.5.
- **Lavorate sui test?** Concentratevi sulla Sezione 9.
- **Volete solo sapere le cose piu' urgenti?** Andate direttamente alla Sezione 10, tabella CRITICO.

---

3. Glossario dei Concetti Fondamentali

Questo glossario spiega i termini tecnici che incontrerete in tutto il documento. Ogni termine e' accompagnato da un'analogia con il mondo reale per rendere il concetto piu' intuitivo. Vi consigliamo di leggere questo glossario per intero prima di procedere, oppure di tornarci ogni volta che incontrate un termine che non conoscete.

Script (.gd)

Un file con estensione `.gd` e' un **file di codice GDScript**, il linguaggio di programmazione di Godot. E' come il foglio delle istruzioni di un mobile IKEA: dice esattamente al computer cosa deve fare, passo dopo passo. Ogni script viene "attaccato" a un nodo nella scena e ne controlla il comportamento.

Esempio: `audio_manager.gd` e' lo script che gestisce tutta la musica e gli effetti sonori del gioco.

Scena (.tscn)

Un file con estensione `.tscn` e' una **scena di Godot**. Pensate a una scena come a un "progetto di costruzione": descrive quali nodi esistono, come sono disposti nello spazio, e quali script hanno attaccati. E' come il progetto architettonico di una stanza: specifica dove va il tavolo, dove va la sedia, le dimensioni, i colori.

Esempio: `main_menu.tscn` descrive la struttura visuale del menu principale — i bottoni, lo sfondo, le animazioni.

Nodo (Node)

Il **nodo** e' l'unita' fondamentale di Godot. Ogni cosa nel gioco e' un nodo: un personaggio, un bottone, un suono, una luce. Pensate ai nodi come a mattoncini LEGO: ognuno ha una funzione specifica (un mattoncino e' una ruota, un altro e' un muro), e combinandoli si costruisce qualcosa di complesso.

Ogni nodo ha un tipo che ne determina le capacita': - `Sprite2D`: puo' mostrare un'immagine - `AudioStreamPlayer`: puo' riprodurre un suono - `Button`: e' un bottone cliccabile - `CollisionShape2D`: definisce una forma per le collisioni

---

Scene Tree (Albero delle Scene)

La **Scene Tree** e' la struttura gerarchica che organizza tutti i nodi del gioco. Funziona come un albero genealogico: ogni nodo ha un "genitore" (parent) e puo' avere dei "figli" (children).

Questa gerarchia e' importante perche': - Quando un nodo genitore viene spostato, anche tutti i figli si spostano. - Quando un nodo genitore viene eliminato, anche tutti i figli vengono eliminati. - I nodi figli possono comunicare con il genitore e viceversa.

Analogia: Pensate a una scrivania (nodo genitore) con sopra un monitor, una tastiera e un mouse (nodi figli). Se spostate la scrivania, tutto cio' che c'e' sopra si sposta insieme.

Autoload / Singleton

Un **Autoload** (detto anche **Singleton**) e' uno script che viene caricato automaticamente all'avvio del gioco e rimane attivo per tutta la durata dell'esecuzione. Non appartiene a nessuna scena specifica: e' disponibile ovunque, in qualsiasi momento.

Analogia: Pensate a un servizio di emergenza come il 112. Non importa dove vi troviate in citta', potete sempre chiamare il 112 e qualcuno rispondera'. Non dovete cercarlo o crearlo: e' sempre li', pronto. Allo stesso modo, `AudioManager` (un autoload) e' sempre disponibile per riprodurre musica, da qualsiasi punto del gioco.

Nel progetto ci sono 8 autoload: `SignalBus`, `GameManager`, `SaveManager`, `LocalDatabase`, `AudioManager`, `SupabaseClient`, `AppLogger`, `PerformanceManager`.

Segnale (Signal)

Un **segnale** e' il sistema di comunicazione tra nodi di Godot. Quando qualcosa succede (un bottone viene premuto, un nemico viene sconfitto, il gioco viene salvato), il nodo responsabile "emette" un segnale. Altri nodi che sono "in ascolto" su quel segnale reagiscono di conseguenza.

Analogia: Immaginate un campanello d'albergo alla reception. Quando un ospite suona il campanello (emette il segnale), il receptionist (nodo in ascolto) si avvicina per aiutare. L'ospite non ha bisogno di conoscere il receptionist: suona il campanello e basta.

Questo e' un pattern molto potente perche' permette ai nodi di comunicare **senza conoscersi direttamente**, il che rende il codice piu' flessibile e facile da modificare.

SignalBus

Il **SignalBus** e' un autoload speciale che funziona come un "hub centrale" per tutti i segnali del gioco. Invece di far comunicare direttamente i nodi tra loro, tutti passano attraverso il SignalBus.

Analogia: Pensate alla centralina di un ufficio postale. Invece di consegnare le lettere a mano porta a porta (comunicazione diretta tra nodi), tutti depositano le lettere all'ufficio postale (SignalBus) che si occupa di instradarle al destinatario giusto. In questo progetto il SignalBus gestisce 21 segnali diversi.

_ready()

`_ready()` e' una funzione speciale che Godot chiama **automaticamente** quando un nodo e' stato completamente caricato e aggiunto alla Scene Tree. E' il posto giusto per inizializzare variabili, connettersi a segnali, e preparare tutto cio' che serve.

Analogia: E' come il primo giorno di lavoro in un nuovo ufficio. Quando arrivate (`_ready()`), vi presentate ai colleghi (connettete i segnali), organizzate la scrivania (inizializzate le variabili), e vi mettete al lavoro.

```
# Questa funzione viene chiamata automaticamente da Godot
# quando il nodo e' pronto
func _ready() -> void:
    # Ci connettiamo al segnale "room_changed" del SignalBus
    SignalBus.room_changed.connect(_on_room_changed)
    # Carichiamo i dati iniziali
    _load_initial_data()
```

_process(delta)

`_process(delta)` e' una funzione che Godot chiama **ogni singolo frame** (fotogramma). Se il gioco gira a 60 FPS (frame per secondo), questa funzione viene chiamata 60 volte al secondo. Il parametro `delta` e' il tempo trascorso dall'ultimo frame, misurato in secondi.

Analogia: E' come un guardiano che fa il giro di ronda regolarmente. Ad ogni giro controlla se c'e' qualcosa di nuovo da gestire.

Attenzione: poiche' viene chiamata cosi' spesso, il codice dentro `_process()` deve essere molto efficiente. Operazioni pesanti qui dentro rallentano tutto il gioco.

_physics_process(delta)

Simile a `_process(delta)`, ma chiamata a **intervalli fissi** (tipicamente 60 volte al secondo, indipendentemente dal framerate). Si usa per tutto cio' che riguarda la fisica: movimenti di personaggi, collisioni, gravita'.

Analogia: Se `_process()` e' un guardiano che fa il giro quando puo', `_physics_process()` e' come il battito di un metronomo: perfettamente regolare, sempre allo stesso ritmo. Questo e' fondamentale per la fisica, dove la regolarita' garantisce movimenti fluidi e prevedibili.

_exit_tree()

`_exit_tree()` e' una funzione che Godot chiama quando un nodo sta per essere **rimosso** dalla Scene Tree. E' il posto dove fare "pulizia": disconnettere i segnali, fermare i timer, liberare le risorse.

Analogia: E' come le procedure di chiusura di un negozio la sera. Prima di uscire, spegnete le luci (fermate i timer), chiudete le casse (disconnettete i segnali), mettete l'allarme (liberate le risorse). Se non fate queste operazioni, lasciate le luci accese tutta la notte (memory leak) e le casse aperte (segnali che puntano a nodi inesistenti, causando crash).

Questo concetto e' FONDAMENTALE per questo audit: molti dei problemi trovati riguardano proprio la mancanza di questa funzione di pulizia.

```
# Questa funzione viene chiamata quando il nodo viene rimosso
func _exit_tree() -> void:
    # Disconnettiamo tutti i segnali a cui ci eravamo connessi
    if SignalBus.room_changed.is_connected(_on_room_changed):
        SignalBus.room_changed.disconnect(_on_room_changed)
    # Fermiamo eventuali tween attivi
    if _tween and _tween.is_running():
        _tween.kill()
```

_input(event)

`_input(event)` e' una funzione che Godot chiama ogni volta che l'utente interagisce con il gioco: preme un tasto, muove il mouse, tocca lo schermo.

Analogia: E' come un receptionist sempre pronto ad ascoltare le richieste dei clienti. Ogni volta che un cliente (l'utente) fa qualcosa (preme un tasto), il receptionist (`_input`) lo nota e decide come rispondere.

---

queue_free()

`queue_free()` e' il comando per **eliminare un nodo in modo sicuro**. Non lo elimina immediatamente, ma lo "mette in coda" per l'eliminazione alla fine del frame corrente. Questo e' importante perche' eliminare un nodo mentre e' ancora in uso causerebbe crash.

Analogia: E' come mettere un pacco nella coda "da spedire" invece di buttarlo dalla finestra. Il pacco verra' gestito correttamente alla fine della giornata lavorativa (fine del frame), senza interrompere il lavoro in corso.

---

call_deferred()

`call_deferred()` ritarda l'esecuzione di una funzione alla **fine del frame corrente**. Si usa quando si vuole fare qualcosa che non puo' essere fatto immediatamente (per esempio, aggiungere un nodo figlio subito dopo averne eliminato un altro).

Analogia: E' come un post-it con scritto "fai questa cosa quando hai finito quello che stai facendo adesso". Non interrompete il lavoro in corso, ma vi assicurate che la cosa venga fatta.

```
# Esempio: eliminare il vecchio personaggio e aggiungere il nuovo
# in modo sicuro, senza conflitti
old_character.queue_free()  # elimina alla fine del frame
call_deferred("add_child", new_character)  # aggiunge dopo l'eliminazione
```

Memory Leak

Un **memory leak** (letteralmente "perdita di memoria") si verifica quando il programma alloca memoria per qualcosa (crea oggetti, carica immagini) ma non la libera mai quando non serve piu'. Col passare del tempo, il programma usa sempre piu' memoria finche' il computer non rallenta o il gioco crasha.

Analogia: Immaginate un rubinetto che perde acqua goccia a goccia. Una goccia non e' un problema. Ma se lasciate il rubinetto cosi' per ore, il lavandino trabocca e il pavimento si allaga. Allo stesso modo, un memory leak piccolo diventa un problema enorme se il gioco resta aperto a lungo.

Nel nostro progetto: Abbiamo trovato diversi memory leak, per esempio i drag preview non eliminati dopo l'uso e i FileDialog creati ad ogni click senza mai distruggerli.

Race Condition

Una **race condition** (condizione di gara) si verifica quando due operazioni cercano di accedere o modificare la stessa risorsa contemporaneamente, e il risultato dipende da quale finisce prima.

Analogia: Immaginate due persone che scrivono sullo stesso foglio di carta contemporaneamente. Il risultato dipende da chi scrive prima, da chi sovrascrive chi, e potrebbe essere un pasticcio illeggibile. Nel software, questo puo' causare dati corrotti, crash, o comportamenti imprevedibili.

Nel nostro progetto: Per esempio, l'auto-save timer potrebbe tentare di salvare il gioco mentre un altro salvataggio e' gia' in corso, corrompendo i dati.

Null

Null e' un valore speciale che significa "niente", "vuoto", "non esiste". Una variabile con valore null non contiene nessun dato utilizzabile.

Analogia: E' come una busta vuota. Se qualcuno vi chiede di leggere il contenuto della busta e la busta e' vuota, non potete farlo. Se il programma prova a usare una variabile null come se contenesse qualcosa, si genera un errore (crash).

Null Check

Un **null check** e' una verifica che si fa prima di usare una variabile per assicurarsi che non sia null. E' una pratica difensiva fondamentale.

Analogia: Prima di aprire una porta, controllate che la porta esista. Sembra ovvio, ma nel software e' facile dimenticarsi di farlo, soprattutto quando i dati arrivano da fonti esterne.

```
# SBAGLIATO: se "texture" e' null, il programma crasha
sprite.texture = texture
sprite.texture.get_size()  # CRASH se texture e' null!

# CORRETTO: controlliamo prima che texture non sia null
if texture != null:
    sprite.texture = texture
    var size = sprite.texture.get_size()
else:
    push_error("Texture non trovata!")
```

Type Hint

Un **type hint** (indicazione di tipo) e' un'annotazione nel codice che specifica quale tipo di dato una variabile deve contenere. In GDScript non sono obbligatori, ma sono fortemente consigliati perche' aiutano a prevenire errori.

Analogia: E' come le etichette sui barattoli in cucina. Se un barattolo e' etichettato "zucchero", sapete cosa c'e' dentro senza dover assaggiare. Se qualcuno ci mette il sale per sbaglio, l'etichetta vi avverte che qualcosa non va.

```
# Senza type hint: non si sa cosa contiene "speed"
var speed = 100

# Con type hint: e' chiaro che speed e' un numero decimale
var speed: float = 100.0

# Con type hint anche per i parametri e il valore di ritorno
func calculate_damage(base: int, multiplier: float) -> int:
    return int(base * multiplier)
```

Dictionary

Un **Dictionary** e' una struttura dati che associa **chiavi** a **valori**. Ogni chiave e' unica e punta a un valore specifico.

Analogia: E' esattamente come un dizionario reale: cercate una parola (la chiave) e trovate la sua definizione (il valore). Oppure pensate a una rubrica telefonica: il nome (chiave) vi porta al numero (valore).

```
# Un dizionario che descrive un personaggio
var character: Dictionary = {
    "nome": "Mario",         # chiave: "nome", valore: "Mario"
    "livello": 5,            # chiave: "livello", valore: 5
    "colore_occhi": "verde"  # chiave: "colore_occhi", valore: "verde"
}

# Accedere a un valore tramite la chiave
print(character["nome"])  # Stampa: Mario
```

Array

Un **Array** e' una lista ordinata di elementi. Ogni elemento ha una posizione numerica (chiamata "indice") che parte da 0.

Analogia: E' come una fila di persone in coda alle poste. La prima persona e' in posizione 0, la seconda in posizione 1, e cosi' via. Potete chiedere "chi c'e' in posizione 3?" e ottenere una risposta.

```
# Un array di nomi di stanze
var rooms: Array = ["soggiorno", "cucina", "camera", "bagno"]

# Accedere per indice (attenzione: parte da 0!)
print(rooms[0])  # Stampa: soggiorno
print(rooms[2])  # Stampa: camera
```

JSON

JSON (JavaScript Object Notation) e' un formato testuale per salvare e scambiare dati strutturati. E' leggibile sia dall'uomo che dal computer, il che lo rende ideale per file di configurazione e dati di gioco.

Analogia: E' come un modulo compilato con campi e valori. Il modulo ha una struttura precisa (nome, cognome, eta') e voi riempite i campi.

```
{
    "nome_stanza": "soggiorno",
    "colore_muri": "#FFE4B5",
    "decorazioni": ["lampada", "tappeto", "quadro"]
}
```

Nel nostro progetto, i cataloghi delle stanze, decorazioni, personaggi e tracce musicali sono tutti salvati come file JSON.

SQLite

SQLite e' un database leggero che vive interamente in un singolo file sul disco. Non richiede un server separato (a differenza di MySQL o PostgreSQL), il che lo rende perfetto per applicazioni desktop e giochi.

Analogia: Se un database server come PostgreSQL e' un grande archivio aziendale con un archivista dedicato, SQLite e' un quaderno organizzato che vi portate nello zaino. Piu' semplice, portatile, ma perfettamente funzionale per le vostre esigenze.

Nel nostro progetto, SQLite viene usato come "mirror" (copia speculare) dei dati JSON per avere un fallback in caso di problemi con il file principale.

Schema (Database)

Lo **schema** di un database e' la sua struttura: definisce quali tabelle esistono, quali colonne ha ogni tabella, che tipo di dati contiene ogni colonna, e come le tabelle sono collegate tra loro.

Analogia: E' come la planimetria di un edificio. Non vi dice chi abita dove, ma vi dice quante stanze ci sono, quanto sono grandi, e dove sono le porte che le collegano.

PRIMARY KEY

La **PRIMARY KEY** (chiave primaria) e' un campo (o combinazione di campi) che identifica in modo **unico** ogni riga di una tabella. Non possono esistere due righe con la stessa chiave primaria.

Analogia: E' come il codice fiscale per le persone o il numero di targa per le auto. Nessuno ne ha uno uguale, e serve proprio per identificarvi senza ambiguita'.

Problema trovato nel progetto: La tabella `characters` usa `account_id` come PRIMARY KEY, il che significa che ogni account puo' avere UN SOLO personaggio. E' come se ogni persona potesse avere una sola auto: un limite artificiale causato da un errore nello schema.

FOREIGN KEY

Una **FOREIGN KEY** (chiave esterna) e' un campo in una tabella che fa riferimento alla PRIMARY KEY di un'altra tabella. Serve per creare relazioni tra le tabelle e garantire l'integrita' dei dati.

Analogia: E' come un riferimento in una lettera. Se nella vostra lettera scrivete "come discusso nella riunione del 15 Marzo (verbale n.42)", il numero 42 e' una "chiave esterna" che punta a un documento specifico. Se quel verbale non esiste, il riferimento e' sbagliato.

Problema trovato nel progetto: La tabella `inventario` ha un campo `item_id` che DOVREBBE essere una FOREIGN KEY verso la tabella `items`, ma questa relazione non e' stata definita. Significa che si possono inserire oggetti nell'inventario con ID che non esistono nel catalogo.

Migrazione

Una **migrazione** e' il processo di aggiornare la struttura del database quando il gioco viene aggiornato. Per esempio, se in una nuova versione volete aggiungere un campo "livello_felicita" al personaggio, dovete fare una migrazione che aggiunge quella colonna alla tabella.

Analogia: E' come ristrutturare un appartamento. I mobili (dati) ci sono gia': dovete aggiungere una stanza (nuova tabella) o allargare una porta (modificare una colonna) senza distruggere nulla di quello che c'e' gia'.

Tween

Un **Tween** e' un sistema di Godot per creare animazioni fluide tra due valori. Invece di cambiare un valore bruscamente (per esempio, la posizione di un oggetto da 0 a 100 in un istante), il tween lo cambia gradualmente nel tempo.

Analogia: E' come una dissolvenza cinematografica. Invece di tagliare bruscamente da una scena all'altra, la transizione e' graduale e fluida.

```
# Esempio: spostare un pannello da fuori schermo alla posizione finale
# in 0.3 secondi con una curva di decelerazione
var tween := create_tween()
tween.tween_property(
    panel,       # l'oggetto da animare
    "position",  # la proprietà da modificare
    Vector2(100, 200),  # il valore finale
    0.3          # la durata in secondi
).set_ease(Tween.EASE_OUT)  # decelerazione graduale
```

Crossfade

Il **crossfade** e' una tecnica audio in cui una traccia musicale sfuma gradualmente mentre un'altra entra gradualmente. Per un breve momento, entrambe le tracce sono udibili contemporaneamente.

Analogia: E' come quando un DJ in discoteca fa la transizione tra due canzoni. La prima si abbassa di volume mentre la seconda si alza, creando una transizione fluida senza momenti di silenzio.

Viewport

Il **Viewport** e' l'area visibile del gioco, cioe' la "finestra" attraverso cui il giocatore vede il mondo di gioco. Ha una dimensione in pixel e determina cosa viene mostrato sullo schermo.

Nel progetto Mini Cozy Room, il viewport e' 1280x720 pixel con stretch mode `canvas_items`, il che significa che il contenuto viene ridimensionato per adattarsi alla finestra mantenendo le proporzioni.

Texture Filter (Nearest/Linear)

Il **filtro texture** determina come Godot disegna un'immagine quando viene ingrandita o rimpicciolita.

- **Nearest**: Ogni pixel viene mostrato come un quadratino netto, senza sfumature. Ideale per pixel art.
- **Linear**: I pixel vengono sfumati tra loro per un aspetto piu' morbido. Ideale per grafica HD.

Analogia: Pensate a quando ingrandite una foto sul telefono. "Nearest" e' come vedere i singoli quadratini colorati (pixel). "Linear" e' come quando il telefono "inventa" pixel intermedi per rendere l'immagine piu' morbida.

Mini Cozy Room usa il filtro **Nearest** perche' ha uno stile pixel art.

Pixel Art

Il **pixel art** e' uno stile grafico in cui ogni singolo pixel dell'immagine e' posizionato intenzionalmente dall'artista. Le immagini sono tipicamente piccole (16x16, 32x32, 64x64 pixel) e vengono poi ingrandite nel gioco.

Analogia: E' come un mosaico: ogni tessera (pixel) e' scelta e posizionata con cura per creare l'immagine finale.

Sprite

Uno **Sprite** e' un'immagine 2D che rappresenta un personaggio, un oggetto, o un elemento del gioco. E' l'unita' visiva fondamentale dei giochi 2D.

Analogia: E' come una figurina ritagliata che si muove sul palco di un teatrino.

Spritesheet

Uno **spritesheet** e' un'unica immagine che contiene tutti i frame (fotogrammi) di un'animazione. Invece di avere 10 file separati per 10 frame di camminata, li si mette tutti in un'unica immagine ordinata in griglia.

Analogia: E' come una striscia di pellicola cinematografica: tanti fotogrammi uno accanto all'altro. Il motore di gioco "ritaglia" il fotogramma giusto al momento giusto per creare l'animazione.

CollisionShape

Un **CollisionShape** e' una forma geometrica invisibile (rettangolo, cerchio, capsula) che viene usata dal motore fisico per rilevare le collisioni tra oggetti. Non si vede durante il gioco, ma determina quando due oggetti si "toccano".

Analogia: E' come la sagoma invisibile intorno a un personaggio che determina quando lo "colpite". Puo' essere piu' grande o piu' piccola dell'immagine visibile.

CanvasLayer

Un **CanvasLayer** e' un livello di rendering separato. Permette di disegnare elementi su un "piano" diverso dal gioco principale. Tipicamente, l'interfaccia utente (bottoni, barre della vita, inventario) sta su un CanvasLayer separato che non si muove con la camera del gioco.

Analogia: E' come una pellicola trasparente sovrapposta a un disegno. Il disegno sotto (il gioco) puo' muoversi e scorrere, ma cio' che e' sulla pellicola (la UI) resta fermo al suo posto.

Export

L'**export** e' il processo di compilazione del gioco per una piattaforma specifica (Windows, Mac, Linux, Web). Trasforma il progetto Godot in un file eseguibile che gli utenti possono usare senza avere Godot installato.

Analogia: E' come cuocere una torta. Avete tutti gli ingredienti (il codice sorgente) nel vostro progetto. L'export e' il forno che trasforma gli ingredienti in una torta finita (l'eseguibile) che gli altri possono mangiare (usare).

CI/CD

CI/CD sta per **Continuous Integration / Continuous Deployment**. E' un sistema automatizzato che, ad ogni modifica del codice (commit), esegue automaticamente test, controlli di qualita', e compilazione del progetto.

Analogia: E' come avere un assistente instancabile che, ogni volta che scrivete una pagina del vostro libro, la rilegge, controlla la grammatica, verifica i riferimenti, e prepara una copia pulita. Se trova errori, vi avvisa subito.

Nel progetto Mini Cozy Room, ci sono 3 workflow CI: - `ci.yml`: controlla lo stile del codice e esegue i test - `build.yml`: compila il gioco per Windows e HTML5 - `database-ci.yml`: verifica il database SQLite e PostgreSQL

GdUnit4

GdUnit4 e' un framework di testing specifico per Godot. Fornisce strumenti per scrivere e eseguire test automatizzati sul vostro codice GDScript.

Analogia: E' come un collaudatore in fabbrica che testa ogni pezzo prodotto per assicurarsi che funzioni correttamente prima di spedirlo.

Test Unitario

Un **test unitario** e' un piccolo programma che verifica automaticamente che una singola funzione del vostro codice faccia esattamente quello che deve fare. "Unitario" perche' testa una "unita'" (una funzione) alla volta.

Analogia: E' come un compito in classe con la soluzione gia' pronta. Scrivete la risposta (il codice della funzione) e poi il test verifica se la vostra risposta corrisponde a quella corretta. Se non corrisponde, il test "fallisce" e vi dice dove avete sbagliato.

```
# Esempio di test unitario
func test_array_to_vec2() -> void:
    # Prepariamo l'input
    var input: Array = [10.0, 20.0]
    # Chiamiamo la funzione da testare
    var result: Vector2 = Helpers.array_to_vec2(input)
    # Verifichiamo che il risultato sia quello atteso
    assert_eq(result.x, 10.0)  # x deve essere 10
    assert_eq(result.y, 20.0)  # y deve essere 20
```

---

gdlint / gdformat

gdlint e **gdformat** sono strumenti che controllano lo stile e la formattazione del codice GDScript.

- **gdlint**: analizza il codice e segnala problemi di stile (variabili con nomi poco chiari, funzioni troppo lunghe, ecc.)
- **gdformat**: formatta automaticamente il codice in modo consistente (indentazione, spazi, allineamento)

Analogia: gdlint e' come un correttore di bozze che sottolinea frasi sgrammaticate. gdformat e' come l'impaginatore che allinea il testo e i margini.

Drag-and-Drop

Il **Drag-and-Drop** (trascina e rilascia) e' un'interazione in cui l'utente clicca su un oggetto, lo trascina muovendo il mouse, e lo rilascia nella posizione desiderata.

Nel progetto Mini Cozy Room, il Drag-and-Drop viene usato per piazzare decorazioni nella stanza: l'utente trascina una decorazione dal catalogo e la rilascia nella posizione desiderata.

Callback

Un **callback** e' una funzione che viene passata come argomento a un'altra funzione, per essere "richiamata" (called back) quando succede qualcosa.

Analogia: E' come lasciare il vostro numero di telefono a un ristorante quando il tavolo non e' pronto. Quando il tavolo si libera, vi "richiamano" (callback) per avvisarvi.

Coupling

Il **coupling** (accoppiamento) misura quanto due parti del codice dipendono l'una dall'altra. Un coupling alto significa che modificare un componente richiede di modificare anche altri componenti.

Analogia: Pensate a due scalatori legati dalla stessa corda corta (coupling alto): se uno cade, trascina anche l'altro. Se invece usano corde indipendenti con punti di ancoraggio separati (coupling basso), un problema di uno non influenza l'altro.

Nel progetto: Diversi autoload scrivono direttamente nei dati di altri autoload (per esempio, `AudioManager` scrive in `SaveManager.settings`). Questo e' un coupling alto che rende il codice fragile.

Architettura Signal-Driven

L'**architettura signal-driven** e' un pattern di design in cui i componenti comunicano **esclusivamente** tramite segnali, senza mai chiamarsi direttamente. Questo riduce il coupling e rende ogni componente indipendente e riutilizzabile.

Analogia: E' come una redazione giornalistica moderna. I giornalisti non portano fisicamente gli articoli al tipografo: li pubblicano su un sistema condiviso (segnale), e chi ne ha bisogno (layout, stampa, web) li prende da li'. Nessuno conosce o dipende direttamente dagli altri.

Dirty Flag

Un **dirty flag** (bandierina "sporco") e' una variabile booleana (vero/falso) che indica se ci sono modifiche non ancora salvate.

Analogia: E' come l'asterisco (*) che appare nel titolo di un documento Word quando avete modifiche non salvate. Vi ricorda che dovete salvare prima di chiudere.

Backup

Un **backup** e' una copia di sicurezza dei dati. Se il file originale viene corrotto o cancellato, il backup permette di recuperare i dati.

Analogia: E' come fare una fotocopia di un documento importante e tenerla in un cassetto separato. Se il documento originale si rovina, avete la fotocopia.

Problema trovato nel progetto: Il sistema di backup del salvataggio non verifica se la copia e' andata a buon fine. E' come fare la fotocopia senza controllare che sia uscita: potreste pensare di avere un backup quando in realta' non ce l'avete.

Seed Data

I **seed data** (dati seme) sono dati iniziali che vengono inseriti nel database al momento della sua creazione. Servono per avere un punto di partenza funzionale.

Analogia: E' come arredare una casa appena costruita con i mobili base (letto, tavolo, sedie) prima che gli inquilini si trasferiscano. Senza seed data, il database e' una casa vuota senza nemmeno il letto.

Problema trovato nel progetto: Le tabelle `colore`, `categoria` e `shop` vengono create ma lasciate vuote: nessun dato iniziale viene inserito.

Silent Failure

Un **silent failure** (fallimento silenzioso) si verifica quando qualcosa va storto nel programma ma nessun messaggio di errore viene mostrato. Il programma sembra funzionare normalmente, ma in realta' sta producendo risultati sbagliati o incompleti.

Analogia: E' come una lettera persa dall'ufficio postale che non vi viene mai comunicata. Voi pensate che la lettera sia arrivata, il destinatario non l'ha mai ricevuta, e nessuno dei due lo sa.

Nel progetto: Diversi metodi del database ritornano `false` o un array vuoto sia quando non ci sono risultati sia quando c'e' un errore. E' impossibile distinguere "nessun dato trovato" da "il database e' rotto".

Bounds Check

Un **bounds check** (controllo dei limiti) e' una verifica che un indice sia valido prima di accedere a un elemento di un array. Se un array ha 5 elementi (indici 0-4), accedere all'indice 5 causa un errore.

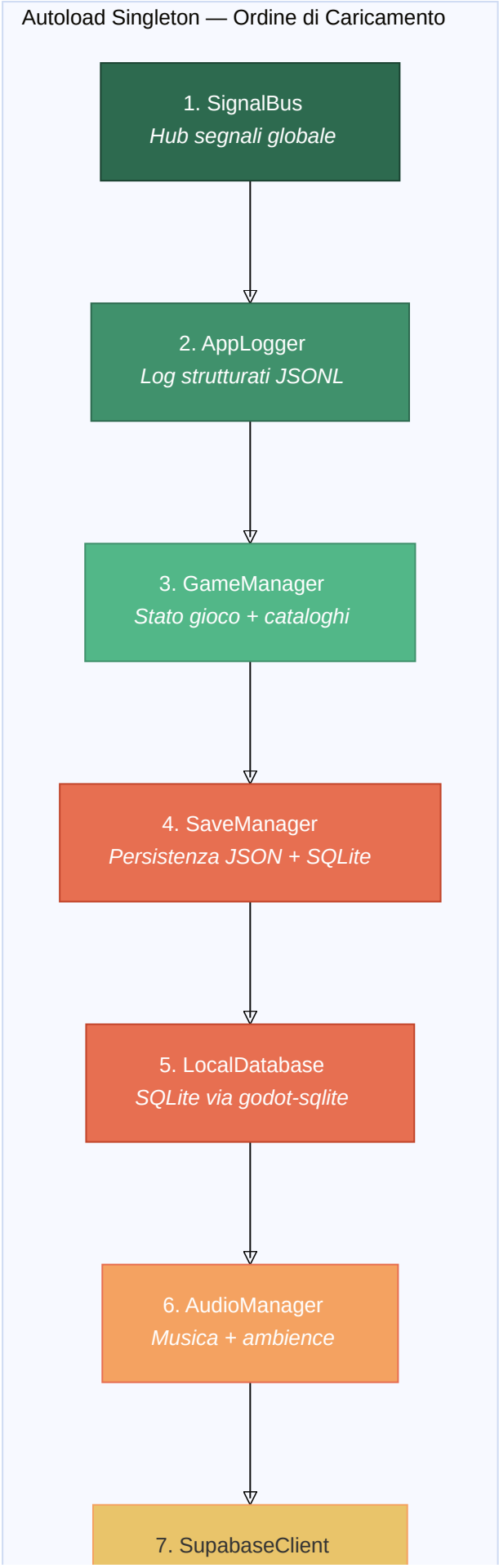
Analogia: Se una fila ha 5 persone, non potete chiedere "chi e' la sesta persona?". Il bounds check e' la verifica che fate prima di chiedere: "ci sono almeno 6 persone in fila?"

```
# SBAGLIATO: se la lista e' vuota, crash!
var first_track = tracks[0]

# CORRETTO: controlliamo prima che la lista non sia vuota
if not tracks.is_empty():
    var first_track = tracks[0]
else:
    push_warning("Nessuna traccia disponibile")
```

---

4. Panoramica del Progetto


Autoload Singleton — Ordine di Caricamento
1. SignalBus
Hub segnali globale
2. AppLogger
Log strutturati JSONL
3. GameManager
Stato gioco + cataloghi
4. SaveManager
Persistenza JSON + SQLite
5. LocalDatabase
SQLite via godot-sqlite
6. AudioManager
Musica + ambience
7. SupabaseClient

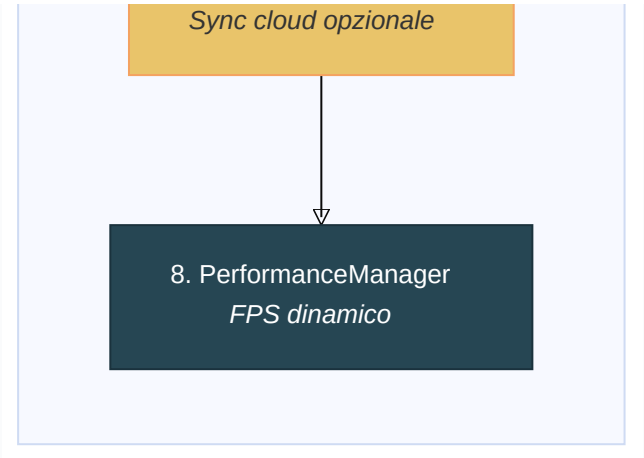


Figura 2 — Architettura Autoload: ordine di caricamento e dipendenze

Cos'e' Mini Cozy Room?

Mini Cozy Room e' un'applicazione desktop companion in 2D. Immaginate un piccolo acquario digitale, ma invece di pesci ci sono personaggi in stile pixel art che vivono in stanze accoglienti. L'utente puo' personalizzare le stanze con decorazioni, cambiare personaggio, ascoltare musica rilassante, e il tutto resta sulla scrivania come un compagno digitale.

Non e' un "gioco" nel senso tradizionale: non ci sono nemici da sconfiggere o livelli da completare. E' un'esperienza rilassante di personalizzazione e compagnia digitale.

Stack Tecnologico

Lo **stack tecnologico** e' l'insieme di tecnologie usate per costruire il progetto. Vediamo ogni componente:

Tecnologia	Cos'e'	Perche' la Usiamo
Godot 4.5	Motore di gioco open source	Gratuito, leggero, perfetto per giochi 2D e applicazioni desktop
GDScript	Linguaggio di programmazione di Godot	Semplice da imparare (simile a Python), integrato nell'editor, ottimizzato per Godot
GL Compatibility	Renderer grafico	Compatibile con la piu' ampia gamma di hardware, ideale per un'app desktop leggera
JSON	Formato dati testuale	Per i cataloghi (stanze, decorazioni, personaggi, tracce musicali)
SQLite	Database leggero in file singolo	Per il backup dei dati di salvataggio, come fallback
Supabase	Backend cloud (opzionale)	Per sincronizzazione dati online — non obbligatorio, il gioco funziona anche offline
GdUnit4	Framework di testing	Per i test automatizzati del codice
GitHub Actions	CI/CD	Per eseguire test e compilazione automaticamente ad ogni commit

Architettura del Progetto

L'architettura di Mini Cozy Room si basa su un pattern chiamato **Signal-Driven Architecture** (architettura guidata dai segnali). Per capirla, usiamo un'analogia.

L'Analogia dell'Ufficio Postale

Immaginate il progetto come un condominio con 8 uffici (gli 8 autoload). Ogni ufficio ha un compito specifico:

Ufficio (Autoload)	Compito
SignalBus	L'ufficio postale centrale — gestisce tutte le comunicazioni
GameManager	Il direttore generale — coordina lo stato del gioco
SaveManager	L'archivista — gestisce il salvataggio e caricamento dei dati
LocalDatabase	Il database locale — gestisce la copia SQLite dei dati
AudioManager	Il tecnico audio — gestisce musica e suoni
SupabaseClient	L'addetto alle comunicazioni esterne — gestisce la connessione al cloud
AppLogger	Il segretario — registra tutto quello che succede (log)
PerformanceManager	Il tecnico della manutenzione — ottimizza le prestazioni

In un'architettura signal-driven ideale, questi uffici **non si parlano direttamente**. Quando il tecnico audio cambia il volume, non va di persona dall'archivista a dirglielo. Invece, lascia un messaggio all'ufficio postale (SignalBus) con scritto "il volume e' cambiato", e l'archivista (SaveManager), che e' abbonato a quel tipo di messaggio, lo riceve e aggiorna i suoi archivi.

Problema trovato: Nel progetto attuale, diversi uffici "scavalcano" l'ufficio postale e vanno direttamente negli altri uffici a modificare i documenti. AudioManager scrive direttamente nei dati di SaveManager, PerformanceManager fa lo stesso, ecc. Questo crea confusione e fragilita'.

Il SignalBus — Il Cuore delle Comunicazioni

Il SignalBus gestisce attualmente **21 segnali** diversi. Ogni segnale rappresenta un tipo di "messaggio" che puo' essere inviato:

- `room_changed`: la stanza e' stata cambiata
- `decoration_placed`: una decorazione e' stata piazzata
- `music_track_changed`: la traccia musicale e' cambiata
- `save_completed`: il salvataggio e' terminato
- ...e molti altri

Ordine di Caricamento degli Autoload

L'ordine in cui gli autoload vengono caricati e' **fondamentale** perche' alcuni dipendono da altri. Godot li carica nell'ordine in cui sono elencati nelle impostazioni del progetto:

1. **SignalBus** — caricato per primo perche' tutti gli altri lo usano per comunicare
2. **AppLogger** — caricato presto perche' gli altri lo usano per registrare messaggi
3. **LocalDatabase** — deve essere pronto prima del SaveManager
4. **SaveManager** — dipende da LocalDatabase per il backup
5. **GameManager** — dipende da SaveManager per caricare lo stato del gioco
6. **AudioManager** — dipende da GameManager per sapere quali tracce caricare
7. **SupabaseClient** — ultimo perche' e' opzionale
8. **PerformanceManager** — ultimo perche' gestisce solo ottimizzazioni

Perche' l'ordine conta: Se il SaveManager prova a usare il LocalDatabase prima che quest'ultimo sia stato caricato, il gioco crasha. E' come cercare di usare il telefono prima che sia stato acceso.

Contenuti del Gioco

Il gioco include: - **4 stanze** con temi diversi e colori personalizzabili - **118 decorazioni** in 14 categorie (mobili, cucina, piante, ecc.) - **3 personaggi** giocabili con animazioni - **2 tracce musicali** (tema pioggia) + sistema di importazione tracce esterne - **FPS dinamico**: 60 FPS

quando il gioco e' in primo piano, 15 FPS quando e' in background (per risparmiare risorse del computer)

---

5. Metodologia di Audit

Come Abbiamo Condotto l'Analisi

L'audit e' stato condotto analizzando **ogni singola riga di codice** di tutti i file del progetto. Non abbiamo usato solo strumenti automatici: ogni file e' stato letto e analizzato manualmente, cercando problemi in diverse aree.

Pensate a questo processo come alla visita medica completa di cui abbiamo parlato nell'introduzione. Non ci siamo limitati a "guardare la facciata": abbiamo controllato ogni organo, ogni funzione vitale.

Aree di Competenza Analizzate

Per ogni area, abbiamo usato come riferimento le guide di best practice (skill files) del progetto:

Area Analizzata	Riferimento	Cosa Abbiamo Cercato
Ciclo di vita dei nodi	`godot-engine-core.md`	I nodi vengono creati e distrutti correttamente? Le risorse vengono liberate?
Qualita' del codice GDScript	`gdscript-mastery.md`	Il codice usa type hints? Gestisce gli errori? E' robusto?
Sistemi 2D	`godot-2d-systems.md`	Sprite, collisioni e trasformazioni sono corretti?
Sistema UI	`godot-ui-ux.md`	I pannelli vengono puliti correttamente? Il drag-and-drop funziona?
Rendering pixel art	`pixel-art-rendering.md`	I filtri texture sono corretti? Lo scaling e' intero?
Audio	`godot-audio.md`	Il crossfade funziona? I volumi sono gestiti in dB?
Persistenza dati	`godot-data-persistence.md`	I salvataggi sono affidabili? SQLite e' usato correttamente? Le migrazioni funzionano?
Testing	`godot-testing.md`	I test coprono le aree critiche? Le asserzioni sono corrette?
Performance	`godot-performance.md`	Il caching e' usato? I tween sono sicuri?
Pattern architetturali	`godot-design-patterns.md`	Il SignalBus e' usato correttamente? Il dirty flag e' implementato?

Criteri di Classificazione

Ogni problema trovato viene classificato secondo la seguente scala di severita'. Ripetiamo la tabella qui per comodita' con esempi concreti dal progetto:

Severita'	Criterio	Esempio dal Progetto
CRITICO	Perdita dati, crash a runtime, vulnerabilita' sicurezza	L'inventario non viene mai salvato su SQLite: se il file JSON si corrompe, tutti gli oggetti dell'utente sono persi per sempre
ALTO	Memory leak, race condition, feature rotta	Il FileDialog viene creato ad ogni click ma mai distrutto: il gioco usa sempre piu' memoria
MEDIO	Validazione mancante, silent failure, code smell	Se una texture non viene trovata, il programma non mostra nessun errore ma il gioco si comporta in modo strano
BASSO	Naming, best practice, ottimizzazione	La costante `CELL_SIZE` e' hardcoded come `64` in piu' file invece di usare una costante condivisa

---

6. Risultati — Autoload Singleton

Gli **autoload** (vedi glossario) sono gli script che formano la "spina dorsale" del progetto. Vengono caricati automaticamente all'avvio del gioco e restano attivi per sempre. Gestiscono le funzionalita' fondamentali: salvataggio, audio, database, comunicazione tra componenti.

Poiche' gli autoload sono sempre attivi e accessibili da qualsiasi punto del gioco, un bug in un autoload ha un impatto potenzialmente **globale**: puo' causare problemi ovunque.

In Mini Cozy Room ci sono 8 autoload. Li analizziamo uno per uno.

6.1 signal_bus.gd (42 righe, 0 funzioni, 21 segnali)

Cosa fa questo file: E' il "centralino telefonico" del gioco (vedi glossario: SignalBus). Contiene solo dichiarazioni di segnali, senza nessuna logica. Quando un componente del gioco vuole comunicare qualcosa (ad esempio "la stanza e' cambiata"), lo fa attraverso questo file.

Stato: BUONO — Il design e' corretto. Un SignalBus deve essere "puro": solo segnali, nessuna logica.

#	Severita'	Problema	Riga	Spiegazione
1	BASSO	TODO non implementato: sistema i18n con segnale `language_changed`	35	Nel codice c'e' un commento TODO (una nota del programmatore che dice "da fare") per aggiungere il supporto multi-lingua. Non e' un bug, ma una funzionalita' prevista e non ancora realizzata.

6.2 game_manager.gd (~130 righe, 12 funzioni)

Cosa fa questo file: E' il "direttore d'orchestra" del gioco. Coordina lo stato generale: quale stanza e' attiva, quale personaggio e' selezionato, quali cataloghi sono caricati. Quando il gioco si avvia, il GameManager carica i dati e mette tutto in moto.

#	Severita'	Problema	Riga	Spiegazione
1	MEDIO	`get_tree().current_scene` puo' essere null — manca controllo null prima di `.scene_file_path`	27-28	`get_tree().current_scene` restituisce la scena attualmente attiva nel gioco. Ma in certi momenti (per esempio durante un cambio di scena), questa puo' essere `null`. Se il codice prova a leggere `.scene_file_path` da qualcosa che e' null, il gioco crasha. La soluzione e' aggiungere un null check prima di usare il valore.
2	MEDIO	`SaveManager.load_game()` chiamato direttamente senza gestione errori	30	Il GameManager chiama `SaveManager.load_game()` ma non controlla se l'operazione e' andata a buon fine. Se il caricamento fallisce (file corrotto, permessi mancanti), il GameManager non lo sa e continua come se tutto fosse OK. E' come spedire una lettera raccomandata senza controllare la ricevuta di ritorno.
3	MEDIO	Sistema outfit personaggio e' placeholder (TODO non implementato)	107	La funzione per cambiare vestiti al personaggio e' solo un segnaposto. Il codice c'e', ma non fa nulla di concreto. Non e' un bug attivo, ma qualsiasi tentativo di cambiare outfit da parte dell'utente non avra' effetto.
4	ALTO	Violazione architetturale: GameManager chiama metodi SaveManager direttamente (coupling bidirezionale)	22, 30, 128	Ricordate l'analogia dell'ufficio postale? Il GameManager sta "scavalcando" il SignalBus e andando direttamente nell'ufficio del SaveManager. Questo crea una dipendenza diretta: se cambiamo il SaveManager, dobbiamo cambiare anche il GameManager. In un'architettura signal-driven, questo non dovrebbe succedere.
5	MEDIO	Cataloghi caricati senza validazione schema — catalogo vuoto `{}` causa errori downstream	33-37	Quando il GameManager carica i cataloghi JSON (stanze, decorazioni, personaggi), non verifica che il contenuto sia corretto. Se un file JSON e' vuoto o malformato, il GameManager lo accetta senza protestare, ma le funzioni che proveranno a usare quei dati piu' tardi falliranno in modo misterioso. E' come accettare un pacco senza controllare il contenuto: se dentro c'e' il prodotto sbagliato, lo scoprirete troppo tardi.

6.3 save_manager.gd (~290 righe, 11 funzioni)

Cosa fa questo file: E' il "custode della memoria" del gioco. Gestisce tutto cio' che riguarda il salvataggio e il caricamento dei dati: la posizione delle decorazioni, le impostazioni audio, lo stato del personaggio, l'inventario degli oggetti. I dati vengono salvati prima come file JSON, poi replicati su SQLite come backup.

Questo file e' **critico** perche' un bug qui significa potenziale **perdita di dati dell'utente**. Se l'utente ha passato ore a decorare la propria stanza e il salvataggio non funziona, tutto quel lavoro va perso.

#	Severita'	Problema	Riga	Spiegazione
1	CRITICO	Race condition: auto-save timer puo' chiamare `save_game()` mentre un salvataggio e' in corso (nessun mutex)	64-67, 70	Il gioco ha un auto-save che salva automaticamente ogni tot secondi. Ma se un salvataggio manuale e' gia' in corso quando il timer scatta, entrambi i salvataggi tentano di scrivere sullo stesso file contemporaneamente. Ricordate l'analogia delle due persone che scrivono sullo stesso foglio? Il risultato puo' essere un file corrotto (dati persi). La soluzione e' aggiungere un flag `_is_saving` che impedisce salvataggi simultanei.
2	CRITICO	Backup file copy non controlla errori — se la copia fallisce, nessun backup esiste	92-93	Quando il gioco salva, prima crea una copia di backup del file precedente (in caso qualcosa vada storto). Ma il codice non verifica se la copia e' riuscita. E' come fare la fotocopia di un documento importante senza controllare che sia uscita dalla fotocopiatrice: potreste pensare di avere un backup quando in realta' non ce l'avete.
3	CRITICO	Inventario MAI salvato su SQLite — `_save_to_sqlite()` salva solo personaggio, non inventario. Dati persi su fallback database	115-120	Questo e' il problema piu' grave trovato nell'intero progetto. La funzione `_save_to_sqlite()` salva i dati del personaggio sul database SQLite (il backup), ma **dimentica completamente l'inventario**. Se il file JSON principale si corrompe e il gioco deve ricorrere al backup SQLite, tutti gli oggetti dell'utente sono persi. E' come se l'archivista facesse la copia di backup del contratto ma dimenticasse tutti gli allegati.
4	ALTO	Se caricamento da file primario E backup falliscono entrambi, `load_completed` emesso senza dati — sistemi a valle non lo sanno	128-129, 202-205	Quando il gioco si avvia, tenta di caricare i dati salvati. Se sia il file principale che il backup sono inaccessibili, il SaveManager emette comunque il segnale "caricamento completato" senza dire a nessuno che i dati sono vuoti. Gli altri sistemi (GameManager, AudioManager) pensano che tutto sia OK e procedono con dati vuoti o corrotti. E' come un corriere che consegna una busta vuota dicendo "ecco il pacco" — il destinatario non sa che manca il contenuto.
5	ALTO	`_compare_versions()` usa `int()` cast senza error handling — versioni non	275-284	La funzione che confronta le versioni del salvataggio (per esempio, "versione 3" vs "versione 4") funziona solo con numeri puri. Se una versione contiene testo (come "1.0.0-

#	Severita'	Problema	Riga	Spiegazione
		numeriche (es. "1.0.0-beta") rompono la comparazione		beta"), la conversione a numero fallisce e il gioco crasha.
6	ALTO	Migrazione v3 verso v4 non valida che la struttura "inventory" sia corretta. Dati malformati passano silenziosamente	245-271	Quando il gioco viene aggiornato, i vecchi salvataggi devono essere "migrati" al nuovo formato. La migrazione dalla versione 3 alla 4 non controlla se i dati dell'inventario hanno la struttura corretta. Dati malformati passano senza nessun errore, causando problemi misteriosi piu' tardi.
7	MEDIO	`FileAccess.open()` fallisce silenziosamente — nessun dirty flag per ritentare	98-101	Se l'apertura del file di salvataggio fallisce (per esempio, perche' il disco e' pieno), il sistema non registra questo fallimento e non prova a ritentare. Il salvataggio semplicemente... non avviene, senza che l'utente lo sappia.
8	MEDIO	File handle leak in `_load_from_file()` — se `json.parse()` fallisce, file non chiuso esplicitamente	150-151	Un "file handle" e' il "canale" attraverso cui il programma legge un file. Se la lettura del JSON fallisce a meta', questo canale resta aperto (non viene chiuso). Pochi file handle aperti non sono un problema, ma molti possono esaurire le risorse del sistema.
9	ALTO	Violazione architetturale: SaveManager chiama `LocalDatabase` e `AudioManager` direttamente	105, 301, 329	Invece di comunicare tramite il SignalBus, il SaveManager chiama direttamente metodi di LocalDatabase e AudioManager. Questo crea dipendenze dirette che rendono il codice piu' difficile da modificare e testare.

6.4 local_database.gd (~282 righe, 21 funzioni)

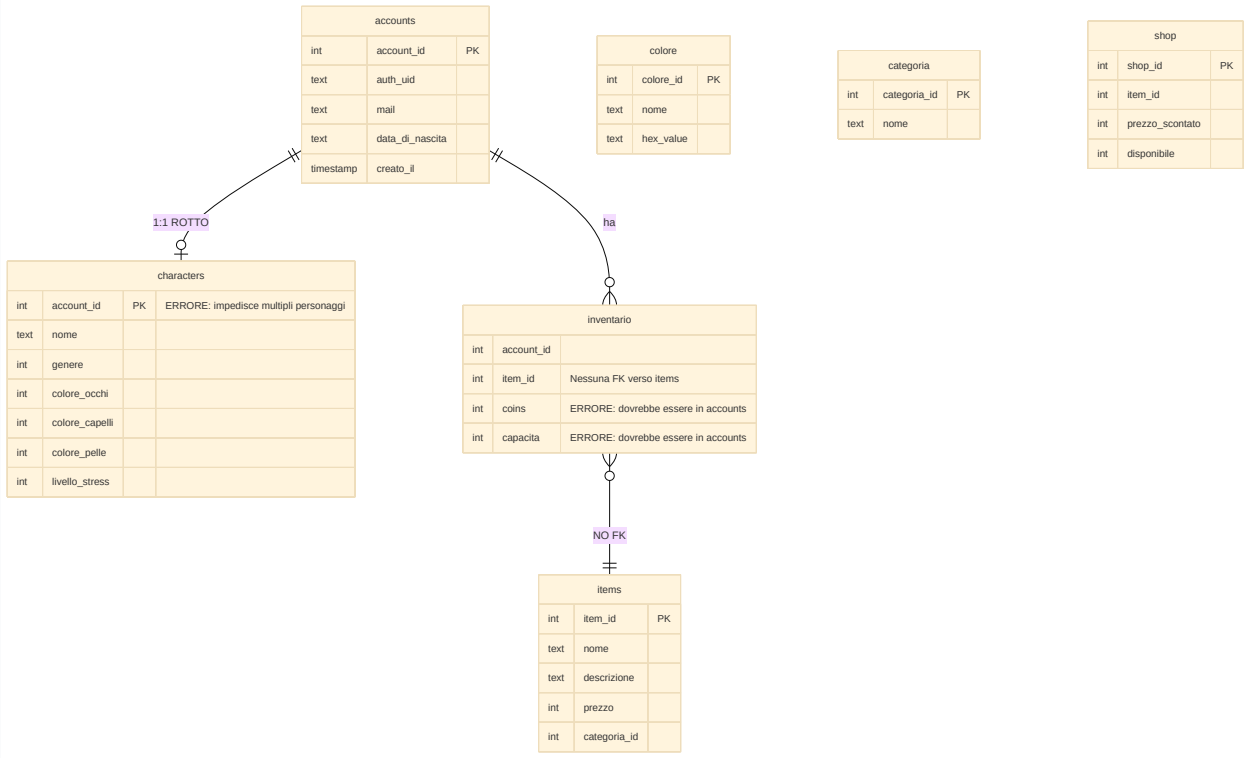


Figura 3 — Schema ER attuale del database (con problemi evidenziati)

Cosa fa questo file: Gestisce il database SQLite locale, che funziona come un "magazzino organizzato" per i dati del gioco. Crea le tabelle (la struttura), inserisce dati, li legge e li aggiorna. Viene usato come backup del salvataggio JSON e potenzialmente per funzionalita' future.

Capire i problemi di questo file richiede una conoscenza base dei database. Brevemente: un database organizza i dati in **tabelle** (pensate a fogli Excel), con **righe** (ogni riga e' un "record", per esempio un personaggio) e **colonne** (le proprieta' di quel record, come nome, livello, colore occhi).

#	Severita'	Problema	Riga	Spiegazione
1	CRITICO	`characters` usa `account_id` come PRIMARY KEY — impossibile avere piu' personaggi per account	101-102	La tabella dei personaggi usa l'ID dell'account come chiave primaria. Poiche' la chiave primaria deve essere unica, questo significa che ogni account puo' avere UN SOLO personaggio. Ma il gioco e' progettato per avere piu' personaggi! E' come se in un'anagrafe il codice della famiglia fosse anche il codice della persona: ogni famiglia potrebbe registrare un solo membro. La soluzione e' usare un `character_id` separato come chiave primaria.
2	CRITICO	Schema `inventario` confuso: `coins` e `capacita` sono per item invece che per account	89-95	Nella tabella dell'inventario, i campi "monete" e "capacita' zaino" sono associati a ogni singolo oggetto invece che all'account dell'utente. E' come se in un supermercato, invece di avere un unico saldo sulla carta fedelta', ogni prodotto nel carrello avesse il suo saldo separato. Non ha senso logico e causa dati incoerenti.
3	ALTO	`item_id` in `inventario` NON e' foreign key verso `items(item_id)` — integrita' referenziale rotta	89	L'inventario contiene oggetti identificati da un `item_id`, ma questo campo non e' collegato alla tabella degli oggetti (`items`). Significa che si possono inserire nell'inventario oggetti con ID inesistenti. E' come poter registrare in biblioteca un prestito per un libro che non esiste nel catalogo.
4	ALTO	`_open_database()` non propaga errori — caller non sa se DB e' aperto	35-42	La funzione che apre il database non comunica al chiamante se l'apertura e' riuscita o meno. Se il file del database e' corrotto o mancante, il resto del codice non lo sa e prova a operare su un database chiuso, causando errori a catena.
5	ALTO	Tabelle `colore`, `categoria`, `shop` create vuote — nessun seed data	48-112	Tre tabelle vengono create con la struttura corretta ma senza nessun dato iniziale. E' come costruire uno scaffale per i libri senza metterci nessun libro. Le funzionalita' che dipendono da queste tabelle (ad esempio, le opzioni di colore per il personaggio) non funzioneranno.

#	Severita'	Problema	Riga	Spiegazione
6	MEDIO	`_execute(sql)` accetta SQL raw senza parametri — potenziale SQL injection se caller passa input utente	251-258	La funzione che esegue comandi SQL li accetta come stringa di testo senza parametri separati. Se un input dell'utente venisse passato direttamente in questa funzione, un utente malintenzionato potrebbe iniettare comandi SQL dannosi. In questo contesto (applicazione desktop locale) il rischio e' basso, ma e' una cattiva pratica.
7	MEDIO	Tutti i query failure ritornano false/array vuoto silenziosamente — impossibile distinguere "nessun risultato" da "errore"	255-280	Quando una query al database fallisce, il codice restituisce `false` o un array vuoto. Ma lo stesso valore viene restituito quando non ci sono risultati. Chi chiama la funzione non puo' sapere se "non ci sono dati" o "c'e' stato un errore". E' come un medico che vi dice "nessun problema" sia quando siete sani sia quando la macchina e' rotta.
8	MEDIO	`upsert_character()` non valida campi richiesti ne' tipi dati del Dictionary input	159-192	La funzione per inserire o aggiornare un personaggio accetta un Dictionary di dati ma non controlla che i campi obbligatori siano presenti ne' che i dati siano del tipo corretto. E' come un modulo che accetta qualsiasi cosa scriviate, anche se nel campo "eta'" mettete "banana".
9	MEDIO	Nessun `delete_inventory_item()` — impossibile rimuovere oggetti dall'inventario	198-210	Esiste una funzione per aggiungere oggetti all'inventario, ma non esiste una funzione per rimuoverli. L'utente puo' collezionare oggetti ma non puo' mai liberarsene.
10	ALTO	Nessun sistema di migrazione schema database — cambiamenti richiedono codice in piu' funzioni	48-112	Non c'e' un sistema strutturato per aggiornare la struttura del database quando il gioco viene aggiornato. Ogni modifica allo schema richiede di cambiare manualmente il codice in diversi punti, con alto rischio di dimenticarne qualcuno.

6.5 audio_manager.gd (~338 righe, 22 funzioni)

Cosa fa questo file: Gestisce tutta la parte audio del gioco: la musica di sottofondo, i suoni ambientali (pioggia, uccelli), il crossfade tra tracce, il volume, e la playlist. E' come il tecnico audio in un teatro: controlla cosa si sente, quando, e a quale volume.

#	Severita'	Problema	Riga	Spiegazione
1	ALTO	Accesso `tracks[current_track_index]` senza bounds check — crash se lista tracce e' vuota	82-85	Il codice accede alla traccia corrente usando un indice, ma non verifica prima che la lista delle tracce contenga almeno un elemento. Se la lista e' vuota (perche' nessuna traccia e' stata caricata), il gioco accede alla posizione 0 di un array vuoto e crasha. Vedi il glossario alla voce "Bounds Check".
2	ALTO	Memory leak ambience: `_start_ambience()` crea AudioStreamPlayer che puo' non essere pulito correttamente con `queue_free()` prima della rimozione dal dizionario	240-245, 270-275	Quando il gioco crea un player per i suoni ambientali, in certi casi il riferimento viene rimosso dal dizionario prima che il player sia effettivamente distrutto. Il player rimane in memoria senza che nessuno ne abbia piu' il riferimento: e' come perdere le chiavi di una macchina parcheggiata — la macchina occupa spazio ma non potete piu' usarla ne' spostarla.
3	ALTO	Crossfade tween kill non garantisce che callback `stop()` del vecchio player venga chiamato — player continua in background	192-194	Durante un crossfade, se il tween (l'animazione di dissolvenza) viene interrotto bruscamente, il callback che dovrebbe fermare il vecchio player audio potrebbe non essere mai chiamato. Il vecchio player continua a suonare in background a volume zero, sprecando risorse.
4	MEDIO	`_load_audio_stream()` per file esterni non ha limite dimensione — potrebbe caricare file enormi in memoria	170-187	Quando l'utente importa una traccia musicale esterna, il gioco la carica interamente in memoria senza controllarne la dimensione. Un file audio di diversi gigabyte potrebbe saturare la memoria del computer.
5	MEDIO	`playlist_mode` non validato — valori invalidi passano silenziosamente nel match statement	61-72	La variabile che controlla la modalita' della playlist (sequenziale, casuale, ripeti) non viene verificata. Se contiene un valore inatteso, il `match` (l'equivalente GDScript dello

#	Severita'	Problema	Riga	Spiegazione
				switch/case) semplicemente non fa nulla, senza segnalare l'errore.
6	MEDIO	Se `_load_audio_stream()` ritorna null, `is_playing` rimane true — stato inconsistente	92-95	Se il caricamento di una traccia audio fallisce, la variabile che indica "sto suonando" resta su `true` anche se in realta' non si sta suonando nulla. L'interfaccia utente mostrerebbe il bottone "pausa" invece di "play", confondendo l'utente.
7	ALTO	Violazione architetturale: `_on_volume_changed()` scrive direttamente in `SaveManager.settings`	292-301	Quando il volume cambia, l'AudioManager va direttamente a modificare i dati interni del SaveManager invece di emettere un segnale. Questo e' come se il tecnico audio andasse nell'ufficio dell'archivista e modificasse i documenti direttamente, senza passare per il protocollo.
8	ALTO	Violazione architetturale: `_sync_music_state()` scrive direttamente in `SaveManager.music_state`	323-329	Stesso problema del punto 7, ma per lo stato della musica (quale traccia e' in riproduzione, a che punto, ecc.).

6.6 supabase_client.gd (~481 righe, 35 funzioni)

Cosa fa questo file: Gestisce la comunicazione con **Supabase**, un servizio cloud che permette di sincronizzare i dati del gioco online. E' l'addetto alle "comunicazioni esterne": si occupa di autenticazione (login/registrazione), invio e ricezione dati dal server, e gestione dei token di sicurezza.

Questo componente e' opzionale — il gioco funziona anche senza connessione internet — ma quando e' attivo, gestisce dati sensibili (credenziali, token di autenticazione), quindi la sicurezza e' particolarmente importante.

#	Severita'	Problema	Riga	Spiegazione
1	CRITICO	Token autenticazione salvati come JSON plaintext in `user://auth.cfg` — vulnerabilita' sicurezza	289-297	I token di autenticazione (le "chiavi" che permettono al gioco di accedere all'account Supabase dell'utente) vengono salvati come testo in chiaro in un file. Chiunque abbia accesso al computer potrebbe leggere questi token e impersonare l'utente. E' come scrivere la password di casa su un biglietto attaccato alla porta. La soluzione e' usare un sistema di cifratura o il keyring del sistema operativo.
2	ALTO	Pool HTTP cresce senza limiti — per sessioni lunghe, memory leak	337-348	Il client Supabase crea connessioni HTTP per comunicare col server. Queste connessioni vengono accumulate in un "pool" (collezione riutilizzabile), ma il pool non ha un limite massimo. Durante sessioni di gioco molto lunghe, il numero di connessioni cresce continuamente senza mai diminuire, consumando sempre piu' memoria.
3	ALTO	Nessun timeout effettivo sulle richieste HTTP — request puo' bloccarsi indefinitamente	332	Quando il gioco invia una richiesta al server Supabase, non c'e' un limite di tempo per la risposta. Se il server non risponde (perche' e' offline o la connessione e' caduta), la richiesta resta in attesa per sempre, potenzialmente bloccando il gioco.
4	ALTO	Token refresh race condition: se token scade durante una richiesta, token stale potrebbe essere inviato	265, 316-318	I token di autenticazione hanno una scadenza. Se un token scade proprio mentre una richiesta e' in corso, la richiesta viene inviata con un token scaduto (stale) e fallisce. Non c'e' un meccanismo per rigenerare il token e ritentare automaticamente.
5	MEDIO	`sign_up()` e `sign_in_email()` non validano email/password vuoti	77-109	Le funzioni di registrazione e login non controllano che l'utente abbia effettivamente inserito email e password. Una richiesta con campi vuoti viene inviata al server, che la rifiutera', ma questo spreca una chiamata di rete e genera un messaggio di errore meno chiaro per l'utente.
6	MEDIO	`query_params` passato direttamente in URL senza encoding — potenziale injection	157, 182, 194	I parametri delle query al database vengono inseriti direttamente nell'URL senza essere "encoded" (convertiti in formato sicuro per URL). Caratteri speciali nei parametri potrebbero essere interpretati come comandi, causando comportamenti imprevisti.

#	Severita'	Problema	Riga	Spiegazione
7	MEDIO	Schema errori inconsistente tra funzioni diverse	77-200	Ogni funzione gestisce gli errori in modo diverso: alcune ritornano un dizionario con un campo "error", altre ritornano `null`, altre emettono segnali. Questa inconsistenza rende difficile per il codice chiamante gestire gli errori in modo uniforme.
8	MEDIO	Nessun meccanismo di retry per richieste fallite	357-371	Se una richiesta al server fallisce (timeout, errore di rete), il codice non prova a ripeterla. In un contesto di rete, dove errori temporanei sono comuni, un meccanismo di retry automatico e' una best practice fondamentale.

6.7 logger.gd (~221 righe, 17 funzioni)

Cosa fa questo file: E' il "segretario" del gioco: registra tutto cio' che succede in un file di log. Ogni operazione importante, ogni errore, ogni avviso viene scritto in un registro che gli sviluppatori possono consultare per diagnosticare problemi. Il logger ha diversi livelli (DEBUG, INFO, WARN, ERROR) che indicano l'importanza di ogni messaggio.

#	Severita'	Problema	Riga	Spiegazione
1	ALTO	`_flush_buffer()` scrive TUTTI i log su disco sincronamente — blocca il game thread se buffer e' grande	121-139	I messaggi di log vengono prima accumulati in un buffer (una coda in memoria) e poi scritti tutti insieme su disco. Ma questa scrittura avviene in modo **sincrono**, cioe' il gioco si ferma e aspetta che tutti i log siano scritti. Se il buffer e' grande (molti messaggi accumulati), il gioco si "congela" per un momento visibile all'utente. E' come un cameriere che serve tutti i piatti in una volta sola: se ha 50 piatti, ci mette un po' e nel frattempo nessun altro viene servito.
2	ALTO	Se file log non puo' essere aperto, buffer viene cancellato — LOG PERSI silenziosamente	125-131	Se il file di log non puo' essere aperto (disco pieno, permessi mancanti), tutti i messaggi nel buffer vengono semplicemente cancellati senza nessun avviso. E' come se il segretario, trovando l'archivio chiuso a chiave, buttasse tutti i documenti nella spazzatura invece di tenerli e riprovare.
3	MEDIO	Session ID generato con `Time ^ PID` — possibili collisioni. Dovrebbe usare UUID	174-185	Ogni sessione di gioco ha un identificatore unico. Ma il metodo di generazione (XOR tra timestamp e Process ID) non e' abbastanza robusto: due sessioni avviate nello stesso secondo potrebbero avere lo stesso ID. Un UUID (Universally Unique Identifier) sarebbe molto piu' sicuro.
4	MEDIO	Timestamp senza millisecondi — log nello stesso secondo hanno timestamp identici	89	I timestamp nei log hanno precisione al secondo. Se due eventi accadono nello stesso secondo (cosa comune in un gioco a 60 FPS), hanno lo stesso timestamp e l'ordine non e' determinabile. Aggiungere i millisecondi risolverebbe il problema.
5	MEDIO	`Level.keys()[level]` assume valore enum valido — crash se fuori bounds	88	Il livello del log (DEBUG=0, INFO=1, WARN=2, ERROR=3) viene usato come indice per ottenere il nome testuale. Ma se il valore e' fuori range (per esempio 5), il codice crasha con un errore di accesso fuori dai limiti dell'array.
6	MEDIO	Nessuna configurazione per log level allo startup — sempre DEBUG di default	19	Il logger parte sempre in modalita' DEBUG, che registra TUTTO, anche messaggi poco importanti. In produzione, sarebbe meglio partire in modalita' WARN o ERROR per ridurre la quantita' di log. Non c'e' modo di configurare il livello iniziale.
7	BASSO	Output console misto: DEBUG/INFO su stdout,	110-118	I messaggi di diverso livello vanno su canali diversi della console. Questo puo' causare un

#	Severita'	Problema	Riga	Spiegazione
		WARN su stderr, ERROR su stderr — ordine inconsistente		ordine di visualizzazione inaspettato quando si leggono i log in tempo reale.

6.8 performance_manager.gd (~55 righe, 6 funzioni)

Cosa fa questo file: Si occupa di ottimizzare le prestazioni del gioco. La sua funzione principale e' il **FPS dinamico**: quando il gioco e' in primo piano (l'utente lo sta usando), gira a 60 FPS per un'esperienza fluida. Quando e' in background (l'utente sta facendo altro), scende a 15 FPS per consumare meno risorse del computer. Gestisce anche il salvataggio della posizione della finestra.

#	Severita'	Problema	Riga	Spiegazione
1	ALTO	Posizione finestra aggiornata in SaveManager.settings ma `save_game()` NON chiamato prima dello shutdown — posizione persa se app crasha	54-55	La posizione della finestra viene scritta nei dati del SaveManager ma non viene effettivamente salvata su disco. Se il gioco crasha o viene chiuso bruscamente, la posizione aggiornata non e' mai stata scritta su file e viene persa. E' come scrivere un appunto su un post-it senza mai attaccarlo nel quaderno: se il post-it cade, l'informazione e' persa.
2	MEDIO	`get_viewport()` puo' essere null — nessun null check prima di connettere segnali	8	Il codice tenta di connettersi ai segnali del viewport senza prima verificare che il viewport esista. In circostanze rare (avvio problematico del gioco), questo potrebbe essere null e causare un crash.
3	MEDIO	Solo posizione X/Y salvata — manca dimensione finestra, stato massimizzato, indice monitor	53-54	Il salvataggio della finestra registra solo la posizione (dove sullo schermo), ma non la dimensione, se era massimizzata, o su quale monitor era. All'avvio successivo, la finestra potrebbe apparire nella posizione giusta ma con la dimensione sbagliata.
4	ALTO	Violazione architetturale: modifica direttamente `SaveManager.settings`	53-54	Come per l'AudioManager, il PerformanceManager scrive direttamente nei dati interni del SaveManager invece di usare il SignalBus.

---

7. Risultati — Script UI, Room e Menu

Questa sezione analizza gli script che gestiscono l'**interfaccia utente** (i pannelli che l'utente vede e con cui interagisce), le **stanze** del gioco (dove vivono i personaggi e le decorazioni), e i **menu** (menu principale, impostazioni).

Questi script sono quelli piu' "visibili" all'utente: un bug qui si manifesta come un pannello che non si chiude, una decorazione che scompare, o il gioco che si blocca durante un'interazione. Molti dei problemi trovati in questa sezione riguardano la **mancanza di pulizia** quando i nodi vengono rimossi (mancanza di `_exit_tree()`), che causa memory leak e crash.

7.1 panel_manager.gd (~136 righe)

Cosa fa questo file: Gestisce l'apertura, la chiusura e l'animazione di tutti i pannelli dell'interfaccia utente (negozio, decorazioni, musica, impostazioni). E' come il responsabile delle porte di un centro commerciale: controlla quale negozio e' aperto, si assicura che non ce ne siano troppi aperti contemporaneamente, e gestisce le animazioni di apertura/chiusura.

#	Severita'	Problema	Riga	Spiegazione
1	ALTO	Nessun `_exit_tree()` — input handler rimane attivo dopo distruzione	132-136	Quando il panel_manager viene distrutto (per esempio, al cambio di scena), i gestori degli input (che intercettano i click e i tasti dell'utente) restano attivi. E' come se le porte del centro commerciale continuassero a funzionare dopo che l'edificio e' stato demolito. Questo puo' causare crash perche' il gestore tenta di operare su nodi che non esistono piu'.
2	MEDIO	Nessun tween null check prima di `_tween.is_running()`	55-58	Prima di controllare se un'animazione (tween) e' in corso, il codice non verifica che il tween esista. Se e' la prima volta che il pannello viene aperto (e il tween non e' ancora stato creato), il codice crasha.
3	MEDIO	Se caricamento scena panel fallisce, ritorna silenziosamente senza feedback	42	Se la scena di un pannello non puo' essere caricata (file mancante o corrotto), la funzione semplicemente ritorna senza dire nulla. L'utente clicca un bottone e non succede niente, senza nessuna spiegazione.

7.2 shop_panel.gd (~160 righe)

Cosa fa questo file: Gestisce il pannello del negozio, dove l'utente puo' sfogliare e acquistare decorazioni per la propria stanza. Mostra le decorazioni organizzate per categoria, con preview (anteprima) e la possibilita' di trascinare gli oggetti nella stanza (drag-and-drop).

#	Severita'	Problema	Riga	Spiegazione
1	ALTO	Preview drag non pulito — memory leak con operazioni drag ripetute	143-152	Ogni volta che l'utente inizia a trascinare un oggetto, il codice crea un'immagine di anteprima (preview) che segue il mouse. Ma quando il drag termina, questa anteprima non viene distrutta. Dopo molti drag-and-drop, centinaia di preview invisibili si accumulano in memoria. E' come se ogni volta che prendete un libro dallo scaffale, una fotocopia apparisse e non venisse mai buttata.
2	ALTO	Nessun `_exit_tree()` — connessioni segnali bottoni non disconnesse	—	I bottoni del negozio (categorie, acquisto) sono connessi a funzioni del pannello. Quando il pannello viene chiuso e distrutto, queste connessioni restano attive, causando potenziali crash e memory leak.
3	MEDIO	`header.text.substr(2)` assume prefisso esatto — fragile se testo modificato	163	Il codice estrae il nome della categoria dal testo dell'header saltando i primi 2 caratteri. Se qualcuno cambia il formato del testo (per esempio aggiungendo un'emoji all'inizio), questo codice si rompe. E' un approccio "fragile" perche' dipende da un'assunzione non documentata.
4	MEDIO	Nessuna gestione errori per dati catalogo corrotti (campi mancanti)	96-104	Quando il pannello mostra le decorazioni, non controlla che i dati nel catalogo siano completi. Se un campo manca (per esempio, il prezzo di un oggetto), il codice crasha o mostra dati sbagliati.

7.3 deco_panel.gd (~200 righe)

Cosa fa questo file: Gestisce il pannello delle decorazioni gia' possedute dall'utente. Mostra le decorazioni nell'inventario e permette di trascinarle nella stanza. E' il complemento del negozio: li' compri, qui usi.

#	Severita'	Problema	Riga	Spiegazione
1	ALTO	Preview drag memory leak (stesso problema di shop_panel)	157-167	Stesso problema del negozio: le anteprime di trascinamento non vengono distrutte. Il problema e' duplicato perche' entrambi i pannelli usano lo stesso pattern di drag-and-drop con lo stesso difetto.
2	MEDIO	`_exit_tree()` e' stub vuoto — segnali bottoni non disconnessi	196-197	La funzione `_exit_tree()` esiste ma e' vuota: non fa nessuna pulizia. E' come avere un cartello "uscita di emergenza" che porta a un muro. La funzione deve essere implementata con la disconnessione dei segnali.
3	MEDIO	Nessun null check su `tex.get_size()` se texture non caricata	157	Se una texture non viene caricata correttamente, il codice tenta comunque di leggerne le dimensioni, causando un crash.

7.4 music_panel.gd (~260 righe)

Cosa fa questo file: Gestisce il pannello musicale, dove l'utente puo' controllare la riproduzione, cambiare traccia, importare nuova musica dal proprio computer, e regolare il volume. E' come il pannello di controllo di un lettore musicale.

#	Severita'	Problema	Riga	Spiegazione
1	CRITICO	`FileDialog` creato e aggiunto alla scena ma MAI rimosso — memory leak accumulativo	236-244	Ogni volta che l'utente clicca "Importa Musica", il codice crea un nuovo FileDialog (la finestra per scegliere il file). Ma il FileDialog precedente non viene mai distrutto. Se l'utente clicca "Importa" 100 volte, ci sono 100 FileDialog in memoria. Questo e' un **memory leak grave e accumulativo**: piu' si usa la funzione, piu' memoria viene consumata, finche' il gioco non rallenta o crasha.
2	ALTO	Solo 2 segnali disconnessi in `_exit_tree()` su ~10 connessi	252-256	La funzione di pulizia disconnette solo 2 dei circa 10 segnali che erano stati connessi. Gli altri 8 restano attivi, puntando a un nodo che non esiste piu'.
3	MEDIO	`AudioManager.tracks[index]` senza bounds check	163	Accesso alla lista delle tracce senza verificare che l'indice sia valido. Se non ci sono tracce, crash.
4	MEDIO	Nessuna validazione campi metadata traccia ("title", "artist")	163	I metadati delle tracce (titolo, artista) non vengono verificati. Se mancano, il pannello mostra dati vuoti o sbagliati.

7.5 settings_panel.gd (~135 righe)

Cosa fa questo file: Gestisce il pannello delle impostazioni, dove l'utente puo' regolare i volumi (musica, effetti, ambiente), scegliere la lingua, e configurare altre preferenze. Contiene slider (barre scorrevoli) e menu a tendina.

#	Severita'	Problema	Riga	Spiegazione
1	ALTO	`_exit_tree()` e' stub vuoto — 4 slider signals e 1 option signal non disconnessi	134-135	Come nel deco_panel, la funzione di pulizia esiste ma non fa nulla. Cinque segnali restano connessi dopo la distruzione del pannello, causando potenziali crash quando gli slider tentano di comunicare con un pannello che non esiste piu'.
2	MEDIO	Scrittura diretta in `SaveManager.settings["language"]` senza validazione	128	Quando l'utente cambia lingua, il codice scrive direttamente nei dati del SaveManager senza verificare che il valore sia una lingua valida. Un valore invalido potrebbe causare problemi nel sistema di traduzione.
3	MEDIO	Race condition: slider `value_changed` puo' attivarsi durante `_load_settings()` nonostante flag `_loading`	97	Quando le impostazioni vengono caricate, gli slider vengono aggiornati ai valori salvati. Ma aggiornare uno slider provoca l'emissione del segnale `value_changed`, che a sua volta tenta di salvare il nuovo valore. C'e' un flag `_loading` per prevenire questo, ma in certe tempistiche la protezione non funziona.

7.6 drop_zone.gd (~80 righe)

Cosa fa questo file: Definisce le "zone di rilascio" dove le decorazioni possono essere piazzate nella stanza. Quando l'utente trascina una decorazione e la rilascia, il drop_zone controlla se la posizione e' valida e piazza l'oggetto.

#	Severita'	Problema	Riga	Spiegazione
1	ALTO	Cast unsafe: `load(sprite_path) as Texture2D` — se risorsa e' tipo diverso, tex e' null e riga 23 crasha	17	Il codice carica una risorsa e la "casta" (converte) a Texture2D. Se la risorsa e' di un tipo diverso (per esempio, un file audio caricato per errore), il cast restituisce null e il codice successivo crasha. E' come prendere un pacco e dare per scontato che contenga un libro: se contiene un vaso, le vostre istruzioni per "leggere il libro" non funzioneranno.
2	MEDIO	`_can_drop_data()` ritorna false silenziosamente — nessun feedback utente	18-19	Quando l'utente trascina una decorazione in una posizione non valida, la funzione rifiuta silenziosamente il rilascio. L'utente non capisce perche' non riesce a piazzare l'oggetto: nessun messaggio, nessuna indicazione visiva.
3	MEDIO	Soglia overlap 50% senza commento — intenzionale o bug?	58	C'e' una soglia del 50% per la sovrapposizione tra decorazioni, ma non c'e' nessun commento che spieghi se e' una scelta di design o un valore arbitrario. Questo rende difficile per altri sviluppatori capire se possono modificarlo.
4	MEDIO	`Helpers.array_to_vec2()` non valida contenuto array	43	La funzione helper che converte un array in un vettore 2D non controlla che i valori nell'array siano numeri validi.

7.7 room_base.gd (~110 righe)

Cosa fa questo file: E' lo script base per tutte le stanze del gioco. Gestisce il caricamento delle decorazioni, il posizionamento del personaggio, e la comunicazione con i pannelli UI. E' come la planimetria della stanza che determina dove va ogni elemento.

#	Severita'	Problema	Riga	Spiegazione
1	ALTO	Nessun `_exit_tree()` — 3 segnali SignalBus non disconnessi, si accumulano al cambio scena	16-18	Quando l'utente cambia stanza, la vecchia stanza viene distrutta. Ma i 3 segnali SignalBus connessi in `_ready()` non vengono disconnessi. Ogni cambio di stanza aggiunge nuovi listener senza rimuovere i vecchi: dopo 10 cambi di stanza, ci sono 30 handler attivi di cui 27 puntano a nodi distrutti.
2	ALTO	Race condition: `queue_free()` del vecchio personaggio + immediato `add_child()` del nuovo — riferimento stale	35-43	Quando l'utente cambia personaggio, il vecchio viene eliminato con `queue_free()` (che agisce a fine frame) e il nuovo viene aggiunto immediatamente. Ma `queue_free()` non elimina subito: per un breve momento, entrambi i personaggi esistono nella scena, il che puo' causare conflitti. La soluzione e' usare `call_deferred("add_child", new_char)` per aggiungere il nuovo personaggio solo dopo che il vecchio e' stato effettivamente rimosso.
3	MEDIO	Position array parsing senza validazione struttura — dati malformati causano crash	75-77	Le posizioni delle decorazioni vengono lette da un array di dati senza verificare che la struttura sia corretta. Se i dati sono malformati (per esempio, un array con un solo elemento invece di due), il parsing crasha.
4	MEDIO	Decorazioni sconosciute logged come warning ma dati persi silenziosamente	72-74	Se il salvataggio contiene una decorazione che non esiste piu' nel catalogo (perche' e' stata rimossa in un aggiornamento), viene loggato un avviso ma la decorazione viene semplicemente ignorata. I dati dell'utente su quella decorazione (posizione, ecc.) vengono persi senza nessuna possibilita' di recupero.

7.8 decoration_system.gd (~70 righe)

Cosa fa questo file: Gestisce il sistema di piazzamento delle decorazioni nella stanza. Controlla il drag-and-drop (trascinamento e rilascio) delle decorazioni, il loro posizionamento, e la rimozione.

#	Severita'	Problema	Riga	Spiegazione
1	ALTO	Se piu' decorazioni hanno stesso `item_id`, solo la prima viene rimossa — dati orfani	64-67	Quando l'utente rimuove una decorazione, il codice cerca la prima decorazione con quell'ID e la rimuove. Ma se ci sono piu' decorazioni uguali (per esempio, due lampade identiche), solo la prima viene rimossa. Le altre restano come "fantasmi" nel sistema dei dati ma senza rappresentazione visiva, creando incoerenze.
2	MEDIO	Nessun `_exit_tree()` — input handler rimane attivo dopo `queue_free()`	10-11	Come negli altri script, manca la pulizia alla distruzione del nodo.
3	MEDIO	Clamp non tiene conto della dimensione sprite — posizione puo' uscire dai limiti visuali	40-43	Il "clamp" (limitazione) della posizione tiene conto solo del centro dell'oggetto, non della sua dimensione. Un oggetto grande puo' avere il centro dentro l'area valida ma i bordi fuori dallo schermo.

7.9 character_controller.gd (~50 righe)

Cosa fa questo file: Controlla il personaggio nella stanza: le animazioni (camminata, interazione, rotazione), il movimento, e gli stati. E' il "burattinaio" che fa muovere il personaggio in base alle azioni dell'utente.

#	Severita'	Problema	Riga	Spiegazione
1	MEDIO	Nessun null check su `_anim` — crash se nodo AnimatedSprite2D non esiste	8, 20-48	La variabile `_anim` dovrebbe puntare al nodo AnimatedSprite2D (il componente che mostra le animazioni del personaggio). Ma non viene mai verificato che questo nodo esista realmente nella scena. Se manca, ogni tentativo di usare `_anim` causa un crash.
2	MEDIO	Nomi animazione hardcoded senza validazione — animazione inesistente viene ignorata silenziosamente	20-48	I nomi delle animazioni (come "walk_down", "idle_side") sono scritti direttamente nel codice. Se un personaggio non ha una di queste animazioni (come nel caso di `male_black_shirt` che ha solo `idle_down`), Godot tenta di riprodurre un'animazione inesistente e silenziosamente non fa nulla. Il personaggio si "congela" senza spiegazione.

7.10 room_grid.gd (~35 righe)

Cosa fa questo file: Disegna la griglia sulla stanza quando l'utente e' in modalita' decorazione. La griglia aiuta a posizionare le decorazioni in modo ordinato, dividendo la stanza in celle quadrate.

#	Severita'	Problema	Riga	Spiegazione
1	MEDIO	Nessun `_exit_tree()` — segnale `decoration_mode_changed` non disconnesso	12	Manca la pulizia del segnale alla distruzione del nodo.
2	BASSO	`CELL_SIZE` hardcoded (64) — dovrebbe usare costante condivisa	5	La dimensione della cella della griglia (64 pixel) e' scritta direttamente nel codice invece di usare la costante `Constants.GRID_CELL_SIZE`. Se la dimensione dovesse cambiare, bisognerebbe modificarla in piu' file.

7.11 window_background.gd (~70 righe)

Cosa fa questo file: Gestisce lo sfondo della finestra del gioco, che e' composto da piu' livelli (layers) con un leggero effetto parallasse (i livelli si muovono a velocita' diverse, creando un senso di profondita').

#	Severita'	Problema	Riga	Spiegazione
1	CRITICO	Mismatch dimensione array: se caricamento layer fallisce, `_layers` e `_parallax_factors` hanno dimensioni diverse — CRASH out-of-bounds	33-49, 64-66	Il codice crea due array paralleli: `_layers` (i livelli grafici) e `_parallax_factors` (le velocita' di movimento). Se un layer non riesce a caricarsi, viene saltato e non aggiunto a `_layers`, ma il fattore di parallasse corrispondente potrebbe comunque essere aggiunto a `_parallax_factors`. I due array finiscono con dimensioni diverse. Quando il codice itera su `_layers` e usa lo stesso indice per accedere a `_parallax_factors`, l'ultimo layer tenta di accedere a un fattore che non esiste: **CRASH**. E' come avere una lista di 3 piatti e una lista di 4 prezzi: quando cercate il prezzo del piatto 4, non lo trovate.
2	MEDIO	Divisione per zero possibile se viewport ha dimensione 0	56-62	L'effetto parallasse richiede di dividere per la dimensione del viewport. Se questa e' 0 (situazione rara ma possibile durante l'inizializzazione), si verifica una divisione per zero.

7.12 main_menu.gd (~110 righe)

Cosa fa questo file: Gestisce il menu principale del gioco — la prima schermata che l'utente vede. Contiene i bottoni per iniziare il gioco, accedere alle impostazioni, e uscire. Include anche la transizione animata dal menu alla stanza di gioco.

#	Severita'	Problema	Riga	Spiegazione
1	ALTO	Nessun `_exit_tree()` — tween e settings panel non puliti al cambio scena	—	Quando l'utente clicca "Gioca" e il gioco passa alla stanza, il menu viene distrutto. Ma i tween attivi (animazioni) e eventuali pannelli impostazioni aperti non vengono puliti, causando memory leak e potenziali crash.
2	MEDIO	`_transitioning` flag senza timeout — se cambio scena fallisce, UI bloccata per sempre	105-110	Quando l'utente avvia la transizione alla stanza, viene impostato un flag che disabilita l'interazione con la UI (per evitare doppi click). Ma se il cambio scena fallisce (file mancante, errore di caricamento), questo flag non viene mai resettato e l'UI resta bloccata per sempre. L'utente non puo' cliccare nulla.
3	MEDIO	Race condition: `load_completed` con `CONNECT_ONE_SHOT` puo' attivarsi dopo cambio scena	63-66	Il menu si connette al segnale "caricamento completato" con la modalita' ONE_SHOT (una sola volta). Ma se il segnale viene emesso dopo che il menu e' gia' stato distrutto (a causa del cambio scena), il callback tenta di operare su un nodo inesistente.

7.13 menu_character.gd (~95 righe)

Cosa fa questo file: Gestisce il personaggio animato che appare nel menu principale. Il personaggio "cammina" sullo schermo con un'animazione di entrata, creando un effetto visivo accogliente.

#	Severita'	Problema	Riga	Spiegazione
1	MEDIO	Timer frame non fermato in `_exit_tree()` — continua dopo distruzione nodo	73-77	Un timer che controlla la velocita' dell'animazione non viene fermato quando il nodo viene distrutto. Il timer continua a "scattare" e tenta di aggiornare un nodo che non esiste piu'.
2	MEDIO	Chiamate multiple a `walk_in()` accumulano sprite — nessun cleanup del precedente	67	Se la funzione `walk_in()` viene chiamata piu' volte (per esempio, ricliccando velocemente), ogni chiamata crea un nuovo sprite senza distruggere il precedente. Dopo 5 chiamate, ci sono 5 personaggi sovrapposti sullo schermo.
3	BASSO	Posizioni walk-in hardcoded (-100 a 640) — non responsive a dimensione viewport	69-70	Le posizioni di partenza e arrivo del personaggio sono numeri fissi scritti nel codice. Se la dimensione della finestra cambia, il personaggio potrebbe partire da fuori schermo troppo lontano o non arrivare al centro.

7.14 main.gd (~70 righe)

Cosa fa questo file: E' lo script della scena principale del gioco (non il menu, ma la schermata di gioco vera e propria). Gestisce il caricamento della stanza corrente, il colore dei muri, e la coordinazione generale tra i vari sistemi.

#	Severita'	Problema	Riga	Spiegazione
1	ALTO	Nessun `_exit_tree()` — segnale `room_changed` non disconnesso	24	Se il gioco torna al menu principale, il segnale `room_changed` resta connesso a una funzione di un nodo distrutto.
2	MEDIO	`Color(wall_hex)` senza validazione formato hex — crash se hex invalido	54-65	Il colore dei muri viene letto come stringa esadecimale (per esempio, "#FFE4B5") e convertito in un oggetto Color. Se la stringa non e' un colore hex valido (per esempio, "non_un_colore"), la conversione crasha.
3	MEDIO	Nessun null check su GameManager	25, 51	Il codice usa il GameManager senza verificare che sia disponibile. In situazioni anomale di avvio, potrebbe essere null.

7.15 constants.gd, helpers.gd, env_loader.gd

Cosa fanno questi file: Sono file di utilita'. `constants.gd` definisce valori costanti usati in tutto il progetto (percorsi, dimensioni, colori). `helpers.gd` contiene funzioni di utilita' generiche (conversioni, formattazioni). `env_loader.gd` carica le variabili d'ambiente (chiavi API, configurazioni).

#	File	Severita'	Problema	Spiegazione
1	constants.gd	MEDIO	`male_black_shirt` definito ma nessuna scena corrispondente in `CHARACTER_SCENES`	Il catalogo dei personaggi lista un personaggio che non ha una scena corrispondente. Se selezionato, il gioco tenta di caricare una scena inesistente.
2	constants.gd	BASSO	Nessuna costante `GRID_CELL_SIZE` — hardcoded come 64 in piu' file	La dimensione della griglia e' un "numero magico" (un valore numerico senza nome) ripetuto in piu' punti del codice. Se va cambiato, bisogna cercarlo e modificarlo ovunque.
3	helpers.gd	MEDIO	`array_to_vec2()` non valida tipo contenuto — valori non numerici causano errore float()	La funzione converte un array in un vettore 2D ma non controlla che i valori siano numeri. Se l'array contiene testo, la conversione a float crasha.
4	helpers.gd	BASSO	Mancano type hints espliciti sui return delle funzioni	Le funzioni non dichiarano il tipo del valore che restituiscono, rendendo il codice meno leggibile e piu' soggetto a errori.
5	env_loader.gd	MEDIO	`get_value()` ricarica file config ad ogni chiamata — inefficiente (riga 47)	Ogni volta che viene richiesto un valore, il file di configurazione viene riletto da disco. Se il valore viene richiesto spesso (per esempio in un loop), questo causa letture disco ripetute e inutili. La soluzione e' caricare il file una volta sola e tenere i valori in memoria.

8. Risultati — Scene e Dati

Questa sezione analizza i **file di scena** (.tscn) e i **file di dati** (JSON, SQL) del progetto. Le scene definiscono la struttura visiva del gioco: dove sono posizionati i nodi, quali proprieta' hanno, e come sono collegati tra loro. I file di dati contengono i contenuti: le stanze disponibili, le decorazioni acquistabili, i personaggi giocabili, le tracce musicali.

Un errore nei dati puo' essere tanto grave quanto un errore nel codice: un percorso file sbagliato in un JSON causa un crash tanto quanto un bug in uno script.

8.1 Scene (.tscn)

Le scene del progetto sono state analizzate per verificare la correttezza della gerarchia dei nodi, i riferimenti agli script, e le proprieta' configurate.

File	Stato	Note
main.tscn	BUONO	Gerarchia corretta, collisioni definite, UILayer a layer 10
main_menu.tscn	BUONO	Bottoni menu, loading screen z=100, parallax background
male-character.tscn	BUONO	AnimatedSprite2D con SpriteFrames, collision CapsuleShape2D, texture_filter=0 (Nearest, corretto per pixel art)
female-character.tscn	BUONO	Stessa struttura del maschile, animazioni aggiuntive (walk_vertical)
cat_void.tscn	BUONO	Sprite semplice con 5 frame, CircleShape2D
UI panels (4)	BUONO	Struttura minimale, script references validi

Le scene non presentano problemi strutturali. La gerarchia e' corretta, i riferimenti sono validi, e le proprieta' sono coerenti.

8.2 Dati JSON

I file JSON contengono i "cataloghi" del gioco: le liste di stanze, decorazioni, personaggi e tracce musicali che il giocatore puo' utilizzare.

rooms.json — BUONO

Il catalogo delle stanze contiene 4 stanze con temi multipli, colori esadecimali validi, e ID consistenti. Nessun problema trovato.

decorations.json — BUONO

Il catalogo delle decorazioni contiene 118 decorazioni in 14 categorie (136 items totali inclusa la cucina). Tutti i percorsi sprite sono stati verificati come esistenti nel filesystem.

Nota: La scala delle decorazioni da cucina (0.3-0.7) e' diversa da quella dei mobili (3.0). Questo e' intenzionale: le decorazioni da cucina sono piu' piccole per la prospettiva isometrica.

characters.json — PROBLEMI CRITICI

Questo file contiene i dati dei personaggi giocabili (sprite di animazione, metadati). Qui sono stati trovati problemi critici:

#	Severita'	Problema	Spiegazione
1	CRITICO	Typo nel percorso sprite: `male_walk_down_side_sxt.png` dovrebbe essere `male_walk_down_side_sx.png` (riga 49)	C'e' un errore di battitura nel nome del file: "sxt" invece di "sx". Il file corretto esiste sul disco ma il JSON punta al file sbagliato. Quando il gioco tenta di caricare questa animazione, non trova il file e crasha. E' un errore di un singolo carattere che causa un crash completo.
2	CRITICO	`male_black_shirt` ha SOLO animazione `idle_down` — crash se richieste altre animazioni	Il personaggio "male_black_shirt" e' incompleto: ha solo l'animazione di stallo verso il basso. Ma il gioco si aspetta che ogni personaggio abbia tutte le animazioni (camminata in tutte le direzioni, interazione, rotazione). Quando l'utente seleziona questo personaggio e prova a camminare, il gioco cerca un'animazione che non esiste e crasha.
3	MEDIO	Directory `charachters` e' un typo (dovrebbe essere `characters`) — presente in tutto il progetto	Il nome della cartella degli asset dei personaggi ha un errore di battitura: "charachters" invece di "characters". Questo errore e' presente in tutto il progetto (JSON, codice, percorsi file). Non causa crash perche' il typo e' consistente (tutti usano la versione sbagliata), ma e' confondente e dovrebbe essere corretto.

tracks.json — MINORE

Il catalogo delle tracce musicali contiene solo 2 tracce a tema pioggia, meno di quanto descritto nella documentazione (che menziona anche tracce lo-fi). L'array `ambience` (suoni ambientali) e' vuoto. Non sono bug, ma contenuti mancanti.

8.3 Supabase Migration SQL — BUONO

Il file di migrazione SQL per Supabase definisce 7 tabelle con Row Level Security (RLS) policies, foreign keys, e cascading deletes. La struttura e' corretta dal punto di vista tecnico.

Nota: I nomi delle colonne sono un mix di italiano e inglese (per esempio, `creato_il` ma `account_id`). Per coerenza, sarebbe meglio scegliere una lingua e usarla consistentemente.

9. Risultati — Test e CI/CD

Questa sezione analizza la **copertura dei test** e i **workflow di Continuous Integration** del progetto.

I test automatizzati sono una rete di sicurezza: quando modificate il codice, i test vi dicono immediatamente se avete rotto qualcosa. Senza test, l'unico modo per verificare e' provare manualmente — un processo lento, impreciso, e impossibile da scalare.

La CI/CD (vedi glossario) automatizza l'esecuzione dei test ad ogni commit, garantendo che nessuna modifica passi inosservata.

9.1 Copertura Test

Il progetto ha attualmente 5 file di test con un totale di 48 test unitari:

File Test	Modulo Testato	# Test	Copertura	Spiegazione
test_helpers.gd	Helpers utility	10	BUONA — vec2, clamp, format, snap	Testa le funzioni di utilita' generiche: conversione di coordinate, limitazione di valori, formattazione stringhe
test_logger.gd	AppLogger	11	BUONA — session ID, livelli, path	Testa il sistema di logging: generazione ID sessione, corretto funzionamento dei livelli di log, percorsi file
test_save_manager.gd	SaveManager schema	12	BUONA — settings, music state	Testa la struttura dei dati di salvataggio: impostazioni, stato musicale
test_save_manager_state.gd	SaveManager state	9	BUONA — decorations, character, inventory	Testa lo stato del gioco: decorazioni, personaggio, inventario
test_shop_panel.gd	ShopPanel	6	DEBOLE — solo catalogo e segnali	Testa solo il caricamento del catalogo e l'emissione dei segnali, ma non il drag-and-drop ne' il ciclo di vita del pannello

Copertura stimata: 15-20%

Questo significa che solo il 15-20% del codice del progetto e' coperto da test automatizzati. L'obiettivo minimo per un rilascio sicuro e' del 50%.

9.2 Aree NON Testate

Queste aree del progetto non hanno alcun test automatizzato:

Area	Rischio	Spiegazione
AudioManager	ALTO	Il sistema audio (crossfade, playlist, ambience) non ha test. Bug nell'audio possono causare crash, memory leak, e esperienza utente degradata
LocalDatabase	ALTO	Le operazioni CRUD (Create, Read, Update, Delete) e l'integrita' dello schema non sono testate. Bug qui causano perdita o corruzione di dati
GameManager	MEDIO	La gestione dello stato e il caricamento dei cataloghi non sono testati
UI Panels (4)	ALTO	Il ciclo di vita dei pannelli, il drag-and-drop, e le interazioni utente non sono testate. Qui si trovano molti dei memory leak scoperti
Room Logic	ALTO	Il piazzamento delle decorazioni e il cambio di personaggio non sono testati
Scene Loading	MEDIO	L'istanziazione delle scene e le transizioni non sono testate
Supabase Client	MEDIO	Il flusso di autenticazione e la gestione HTTP non sono testati
Performance Manager	BASSO	Il capping degli FPS e il salvataggio della posizione finestra non sono testati

9.3 CI/CD

Il progetto utilizza 3 workflow GitHub Actions:

Workflow	Stato	Problemi
ci.yml (lint + test + security)	BUONO	I file di test non sono lintati da gdformat (cioe', il formattatore controlla gli script ma non i test)
build.yml (Windows + HTML5)	BUONO	Manca code signing per il file .exe (l'eseguibile Windows non e' firmato, il che potrebbe generare avvisi di sicurezza)
database-ci.yml (SQLite + PostgreSQL)	ECCELLENTE	Il parsing tramite regex e' un po' fragile ma funzionale

10. Classificazione dei Problemi

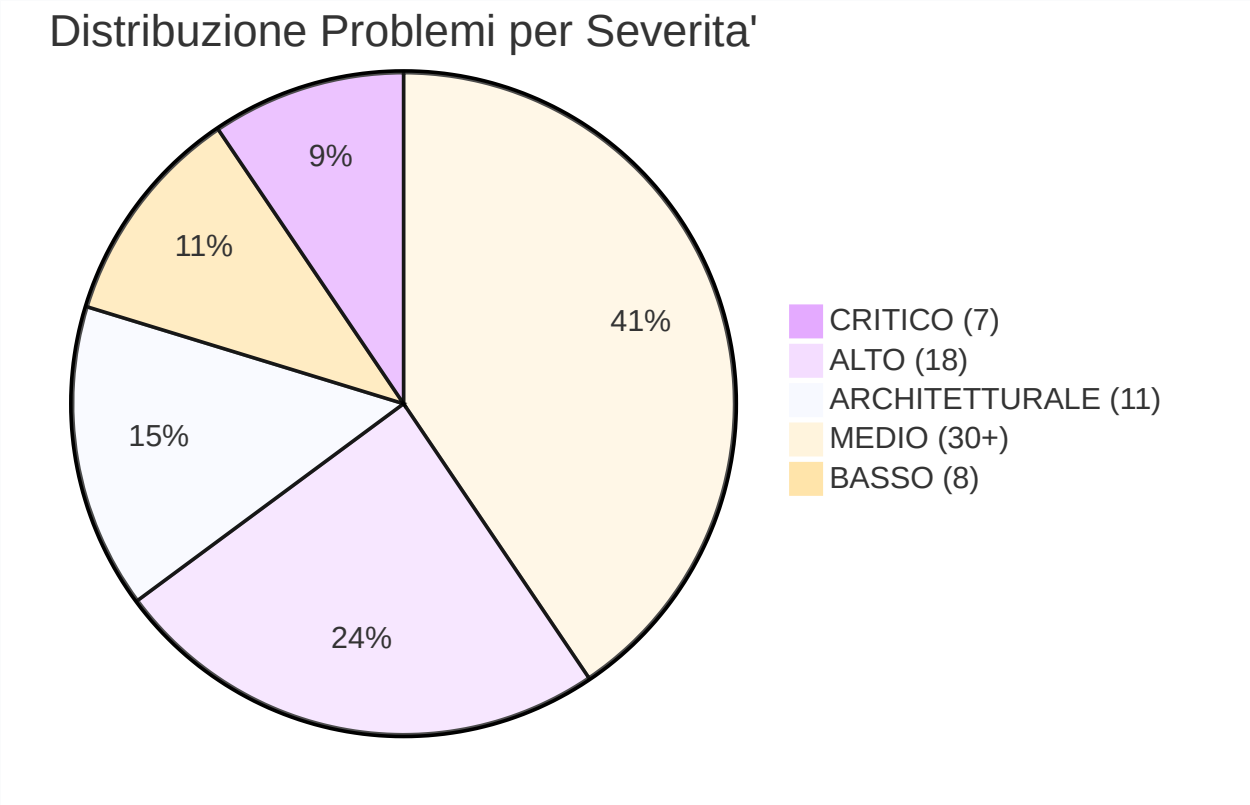


Figura 4 — Distribuzione dei problemi per severita'

Questa sezione presenta tutti i problemi trovati, organizzati per severita' e con un identificatore univoco (C1, A1, AR1...) per riferirsi ad essi facilmente nelle sezioni successive.

CRITICO (7 problemi) — Priorita' Immediata

Questi problemi devono essere risolti **prima di qualsiasi rilascio**. Causano perdita di dati, crash irrecuperabili, o vulnerabilita' di sicurezza.

#	File	Problema	Impatto
C1	save_manager.gd:115	Inventario MAI salvato su SQLite	Perdita dati su fallback DB
C2	save_manager.gd:92	Backup copy senza error checking	Nessun backup se copia fallisce
C3	local_database.gd:101	Characters PK impedisce multipli personaggi	Design schema rotto
C4	local_database.gd:89	Inventory schema confuso (coins per item)	Dati incoerenti
C5	window_background.gd:33	Array mismatch _layers vs _parallax_factors	Crash out-of-bounds
C6	characters.json:49	Typo percorso sprite "sxt" -> "sx"	Crash caricamento animazione
C7	characters.json	male_black_shirt incompleto	Crash cambio animazione

C1 — Inventario MAI salvato su SQLite: La funzione `_save_to_sqlite()` nel SaveManager salva i dati del personaggio sul database SQLite ma dimentica completamente l'inventario. Il database SQLite serve come backup in caso il file JSON principale si corrompa. Se questo succede, l'utente perde tutti i suoi oggetti. Pensate a una banca che fa il backup dei conti correnti ma dimentica di copiare i depositi a risparmio.

C2 — Backup copy senza error checking: Quando il gioco salva, prima crea una copia di backup del salvataggio precedente. Ma il codice non verifica se la copia e' andata a buon fine. Se il disco e' pieno o ci sono problemi di permessi, la copia fallisce silenziosamente e l'utente pensa di avere un backup quando non ce l'ha.

C3 — Characters PK impedisce multipli personaggi: La tabella `characters` nel database usa `account_id` come PRIMARY KEY. Poiche' la chiave primaria deve essere unica, ogni account puo' avere un solo personaggio. Ma il gioco e' progettato per supportare piu' personaggi per account.

C4 — Inventory schema confuso: Nella tabella `inventario`, i campi `coins` (monete) e `capacita` (capacita' zaino) sono associati a ogni singolo oggetto invece che all'account. Questo non ha senso logico e causa dati incoerenti.

C5 — Array mismatch window_background.gd: Se il caricamento di un layer dello sfondo fallisce, gli array `_layers` e `_parallax_factors` finiscono con dimensioni diverse. Quando il codice itera su di essi usando lo stesso indice, si verifica un crash per accesso fuori dai limiti.

C6 — Typo percorso sprite characters.json: Un errore di battitura nel nome del file (`sxt` invece di `sx`) impedisce il caricamento di un'animazione, causando un crash quando il giocatore tenta di muoversi con il personaggio `male_old`.

C7 — male_black_shirt incompleto: Il personaggio `male_black_shirt` ha solo l'animazione `idle_down`. Ogni altro tentativo di animazione (camminata, interazione) crasha perche' le animazioni richieste non esistono.

ALTO (18 problemi) — Prossimo Sprint

Questi problemi non causano perdita di dati immediata, ma degradano l'esperienza utente con memory leak, crash intermittenti, e feature rotte.

#	File	Problema
A1	12 script	Mancanza `_exit_tree()` in: panel_manager, shop_panel, deco_panel, settings_panel, room_base, decoration_system, room_grid, main_menu, menu_character, main, music_panel (parziale), character_controller
A2	music_panel.gd:236	FileDialog memory leak accumulativo
A3	room_base.gd:35	Race condition swap personaggio
A4	audio_manager.gd:240	Memory leak player ambience
A5	audio_manager.gd:82	Crash su lista tracce vuota
A6	shop_panel.gd:143	Memory leak drag preview
A7	deco_panel.gd:157	Memory leak drag preview
A8	save_manager.gd:275	Version comparison rotta per non-numeric
A9	save_manager.gd:245	Migrazione v3 verso v4 non valida struttura
A10	supabase_client.gd:289	Token auth plaintext
A11	supabase_client.gd:337	HTTP pool crescita illimitata
A12	logger.gd:121	Flush sincrono blocca game thread
A13	logger.gd:125	Log persi se file non disponibile
A14	performance_manager.gd:54	Posizione finestra non persistita prima di shutdown
A15	decoration_system.gd:64	Rimozione duplicati item_id rotta
A16	drop_zone.gd:17	Cast Texture2D unsafe
A17	local_database.gd:48	Tabelle seed vuote
A18	local_database.gd:35	Errore apertura DB non propagato

A1 — Mancanza _exit_tree(): Questo e' il problema piu' diffuso nel progetto. 12 script su 26 non implementano correttamente la funzione di pulizia `_exit_tree()`. Questo significa che

quando questi nodi vengono distrutti (per esempio al cambio scena), le connessioni ai segnali restano attive, i timer continuano a scattare, e i tween continuano a funzionare — tutto puntando a nodi che non esistono piu'. Ogni volta che l'utente cambia stanza o chiude un pannello, queste "connessioni fantasma" si accumulano, consumando memoria e rischiando crash. Pensate a dei telefoni che continuano a squillare in uffici dove non lavora piu' nessuno.

A2 — FileDialog memory leak: Ogni click su "Importa Musica" crea un nuovo FileDialog senza distruggere il precedente. 100 click = 100 FileDialog in memoria.

A3 — Race condition swap personaggio: Quando si cambia personaggio, il vecchio viene eliminato con `queue_free()` (che agisce a fine frame) ma il nuovo viene aggiunto immediatamente, creando un conflitto temporaneo.

A4 — Memory leak player ambience: I player audio per i suoni ambientali possono restare in memoria senza riferimenti validi.

A5 — Crash su lista tracce vuota: Se non ci sono tracce musicali caricate, il tentativo di riprodurre causa un crash per accesso a un array vuoto.

A6 e A7 — Memory leak drag preview: Sia nel negozio che nel pannello decorazioni, le anteprime di trascinamento non vengono distrutte.

A8 — Version comparison rotta: La comparazione delle versioni funziona solo con numeri puri (come "3" o "4"). Versioni come "1.0.0-beta" causano crash.

A9 — Migrazione non valida: La migrazione dalla versione 3 alla 4 del salvataggio non verifica l'integrita' dei dati dell'inventario.

A10 — Token auth plaintext: I token di autenticazione Supabase sono salvati in testo chiaro, leggibili da chiunque acceda al computer.

A11 — HTTP pool illimitata: Le connessioni HTTP al server Supabase crescono senza limite durante sessioni lunghe.

A12 — Flush sincrono: La scrittura dei log su disco blocca il gioco, causando "stutter" (piccoli scatti) visibili.

A13 — Log persi: Se il file di log non e' accessibile, tutti i messaggi vengono cancellati senza preavviso.

A14 — Posizione finestra persa: La posizione della finestra viene aggiornata in memoria ma non scritta su disco prima dello shutdown.

A15 — Rimozione duplicati rotta: Se ci sono decorazioni con lo stesso ID, solo la prima viene rimossa correttamente.

A16 — Cast Texture2D unsafe: Il caricamento delle texture non gestisce il caso in cui la risorsa caricata non sia una texture.

A17 — Tabelle seed vuote: Le tabelle di lookup (colore, categoria, shop) sono create senza dati iniziali.

A18 — Errore DB non propagato: Se il database non si apre, il chiamante non lo sa.

ARCHITETTURALE (11 violazioni)

Queste non sono bug che causano crash, ma problemi nella struttura del codice che rendono il progetto difficile da mantenere, testare, e far evolvere. Pensate a fondamenta leggermente storte: la casa sta in piedi, ma aggiungere un piano sara' rischioso.

#	Da -> A	Tipo	Spiegazione
AR1	GameManager -> SaveManager	Chiamata diretta metodo	Il GameManager chiama direttamente le funzioni del SaveManager invece di comunicare tramite segnali. Questo lega i due componenti in modo che modificare uno richiede di modificare anche l'altro.
AR2	SaveManager -> LocalDatabase	Chiamata diretta metodo	Il SaveManager interagisce direttamente col database locale.
AR3	SaveManager -> AudioManager	Chiamata diretta metodo	Il SaveManager chiama direttamente l'AudioManager per sincronizzare lo stato audio.
AR4	AudioManager -> SaveManager.settings	Scrittura diretta dict	L'AudioManager modifica direttamente il dizionario delle impostazioni che appartiene al SaveManager. E' come se un dipendente modificasse i documenti dell'archivio senza passare per l'archivista.
AR5	AudioManager -> SaveManager.music_state	Scrittura diretta dict	Stesso problema di AR4, ma per lo stato della musica.
AR6	PerformanceManager -> SaveManager.settings	Scrittura diretta dict	Il PerformanceManager modifica direttamente le impostazioni del SaveManager.
AR7	settings_panel -> SaveManager.settings	Scrittura diretta dict	Il pannello impostazioni modifica direttamente le impostazioni del SaveManager.
AR8	Autoloads	Nessuna validazione dipendenze cross-autoload in _ready()	Nessun autoload verifica che i suoi autoload dipendenti siano gia' disponibili prima di usarli. Se l'ordine di caricamento cambia, il gioco crasha.
AR9	Tutti i manager	Nessuna propagazione errori (tutto silenzioso)	Quando qualcosa va storto in un manager, l'errore viene ingoiato silenziosamente. I manager che dipendono da quel risultato non sanno che c'e' stato un problema.
AR10	local_database.gd	Nessun sistema migrazione schema	Non esiste un sistema strutturato per aggiornare lo schema del database quando il gioco viene aggiornato. Ogni modifica richiede interventi manuali in piu' punti del codice.

#	Da -> A	Tipo	Spiegazione
AR11	supabase_client.gd	Schema errori inconsistente tra funzioni	Ogni funzione gestisce e ritorna errori in modo diverso, rendendo impossibile un trattamento uniforme degli errori da parte del codice chiamante.

11. Piano di Stabilizzazione

Figura 5 — Le 5 fasi del piano di stabilizzazione

Il piano di stabilizzazione e' diviso in **5 fasi**, ordinate per priorita'. Ogni fase affronta una categoria specifica di problemi. L'ordine e' importante: le fasi successive presuppongono che le precedenti siano state completate.

Fase 1 — Integrita' Dati (CRITICO)

Concetto: Perche' i Dati Sono la Priorita' Numero Uno?

In un'applicazione come Mini Cozy Room, i dati dell'utente (decorazioni piazzate, personaggio scelto, oggetti acquistati, impostazioni) rappresentano ore di interazione. Perdere questi dati e' come cancellare il salvataggio di un gioco dopo 100 ore di gioco: l'utente non vi perdonera'.

Per questo, la prima fase si concentra sull'eliminare OGNI possibile percorso che porta alla perdita di dati o a crash irrecuperabili.

1.1 Correggere characters.json

Obiettivo: Eliminare i crash causati da dati mancanti o errati nei personaggi.

Passo 1 — Correggere il typo del file sprite:

Aprite il file `data/characters.json` e cercate la stringa `male_walk_down_side_sxt.png`.

Prima (codice con errore):

```
"male_walk_down_side_sxt.png"
```

Dopo (codice corretto):

```
"male_walk_down_side_sx.png"
```

Verificate che il file corretto esista sul disco:

```
# Verifica che il file con il nome corretto esista
ls v1/assets/charachters/male/old/male_walk/male_walk_down_side_sx.png
```

Passo 2 — Risolvere male_black_shirt:

Avete tre opzioni. La piu' sicura e':

Opzione A (consigliata): Rimuovere il personaggio dal catalogo fino a quando le sprite non sono pronte. Aprite `data/characters.json` e rimuovete l'intera sezione `male_black_shirt`. Poi aprite `constants.gd` e rimuovete il riferimento corrispondente.

Opzione B: Copiare le sprite di un personaggio completo (per esempio `male_yellow_shirt`) come placeholder temporanei.

Opzione C: Aggiungere un fallback nel controller del personaggio che usa `idle_down` quando un'animazione richiesta non esiste:

```
# character_controller.gd
# Prima di riprodurre un'animazione, verifichiamo che esista
# Se non esiste, usiamo "idle_down" come animazione di sicurezza
if _anim.sprite_frames.has_animation(anim_name):
    _anim.play(anim_name)  # l'animazione esiste, la riproduciamo
else:
    _anim.play("idle_down")  # fallback sicuro — idle_down esiste sempre
```

1.2 Correggere window_background.gd

Obiettivo: Eliminare il crash causato dal mismatch tra gli array `_layers` e `_parallax_factors`.

Il problema: Quando un layer non viene caricato, viene aggiunto il factor ma non il layer (o viceversa), causando array di dimensioni diverse.

La soluzione: Garantire che, per ogni layer, vengano aggiunti ENTRAMBI gli elementi (layer e factor) oppure NESSUNO dei due.

Prima (codice problematico — schema concettuale):

```
# PRIMA: se il caricamento fallisce, _layers salta un elemento
# ma _parallax_factors potrebbe non saltarlo
func _build_layers() -> void:
    for i in LAYER_FILES.size():
        var tex = load(LAYER_FILES[i])
        if tex == null:
            continue  # salta il layer MA il factor potrebbe essere aggiunto altrove!
        # ... aggiunge layer E factor
```

Dopo (codice corretto):

```
# DOPO: se il caricamento fallisce, saltiamo TUTTO per quel layer
# Cosi' gli array restano sempre allineati
func _build_layers() -> void:
    for i: int in LAYER_FILES.size():
        # Tentiamo di caricare la texture del layer
        var tex := load(LAYER_FILES[i]) as Texture2D
        if tex == null:
            # Layer non trovato — logghiamo un avviso per il debugging
            AppLogger.warn("WindowBackground", "Layer non trovato", {"file": LAYER_FILES[
            continue  # saltiamo COMPLETAMENTE questo layer

        # Creiamo lo sprite per il layer
        var sprite := Sprite2D.new()
        sprite.texture = tex  # assegniamo la texture caricata
        sprite.texture_filter = CanvasItem.TEXTURE_FILTER_NEAREST  # pixel art: filtro ne
        add_child(sprite)  # aggiungiamo lo sprite alla scena

        # Aggiungiamo ENTRAMBI gli elementi: layer E factor
        # Cosi' gli array hanno SEMPRE la stessa dimensione
        _layers.append(sprite)
        _base_positions.append(sprite.position)
        # Usiamo il factor corrispondente, con fallback 0.05 se manca
        _parallax_factors.append(DEPTH_FACTORS[i] if i < DEPTH_FACTORS.size() else 0.05)
```

1.3 Correggere save_manager.gd

Obiettivo: Eliminare le race condition nel salvataggio, verificare i backup, e salvare l'inventario su SQLite.

Passo 1 — Aggiungere flag anti-race-condition:

All'inizio del file, dopo le variabili esistenti, aggiungete:

```
# Flag che indica se un salvataggio e' in corso
# Impedisce che due salvataggi avvengano contemporaneamente
var _is_saving: bool = false
```

Poi, nella funzione `save_game()`, avvolgete il contenuto con un controllo:

```
func save_game() -> void:
    # Se un salvataggio e' gia' in corso, usciamo subito
    # Questo previene la race condition con l'auto-save timer
    if _is_saving:
        AppLogger.warn("SaveManager", "Salvataggio gia' in corso, skip")
        return

    _is_saving = true  # segnaliamo che stiamo salvando

    # ... tutto il codice di salvataggio esistente ...

    _is_saving = false  # salvataggio completato, liberiamo il flag
```

Passo 2 — Aggiungere error checking al backup:

Cercate la riga dove viene copiato il file di backup e sostituitela con:

```
# Apriamo la directory del salvataggio
var dir := DirAccess.open("user://")
if dir:
    # Tentiamo di copiare il file di salvataggio come backup
    var err := dir.copy(SAVE_PATH, BACKUP_PATH)
    if err != OK:
        # La copia e' fallita — logghiamo l'errore
        # L'utente deve sapere che non ha un backup
        AppLogger.error("SaveManager", "Backup fallito", {"errore": err})
else:
    # Non riusciamo nemmeno ad accedere alla directory
    AppLogger.error("SaveManager", "Directory user:// non accessibile")
```

Passo 3 — Aggiungere salvataggio inventario su SQLite:

Nella funzione `_save_to_sqlite()`, dopo la chiamata a `upsert_character()`, aggiungete il salvataggio dell'inventario:

```
# Dopo aver salvato il personaggio, salviamo anche l'inventario
# ATTENZIONE: Prima di usare questo codice, correggere lo schema (C4)
if not inventory_data.get("items", []).is_empty():
    # Per ogni oggetto nell'inventario
    for item: Dictionary in inventory_data["items"]:
        # Aggiungiamo l'oggetto al database
        LocalDatabase.add_inventory_item(
            1,  # ID account locale (fisso per gioco single-player)
            item.get("item_id", 0),  # ID dell'oggetto, default 0 se mancante
            inventory_data.get("coins", 0),  # Monete dell'utente
            inventory_data.get("capacity", 50)  # Capacita' inventario
        )
```

Nota importante: Questo passo richiede che lo schema dell'inventario sia stato prima corretto (vedi C4 nella Sezione 12).

1.4 Correggere local_database.gd

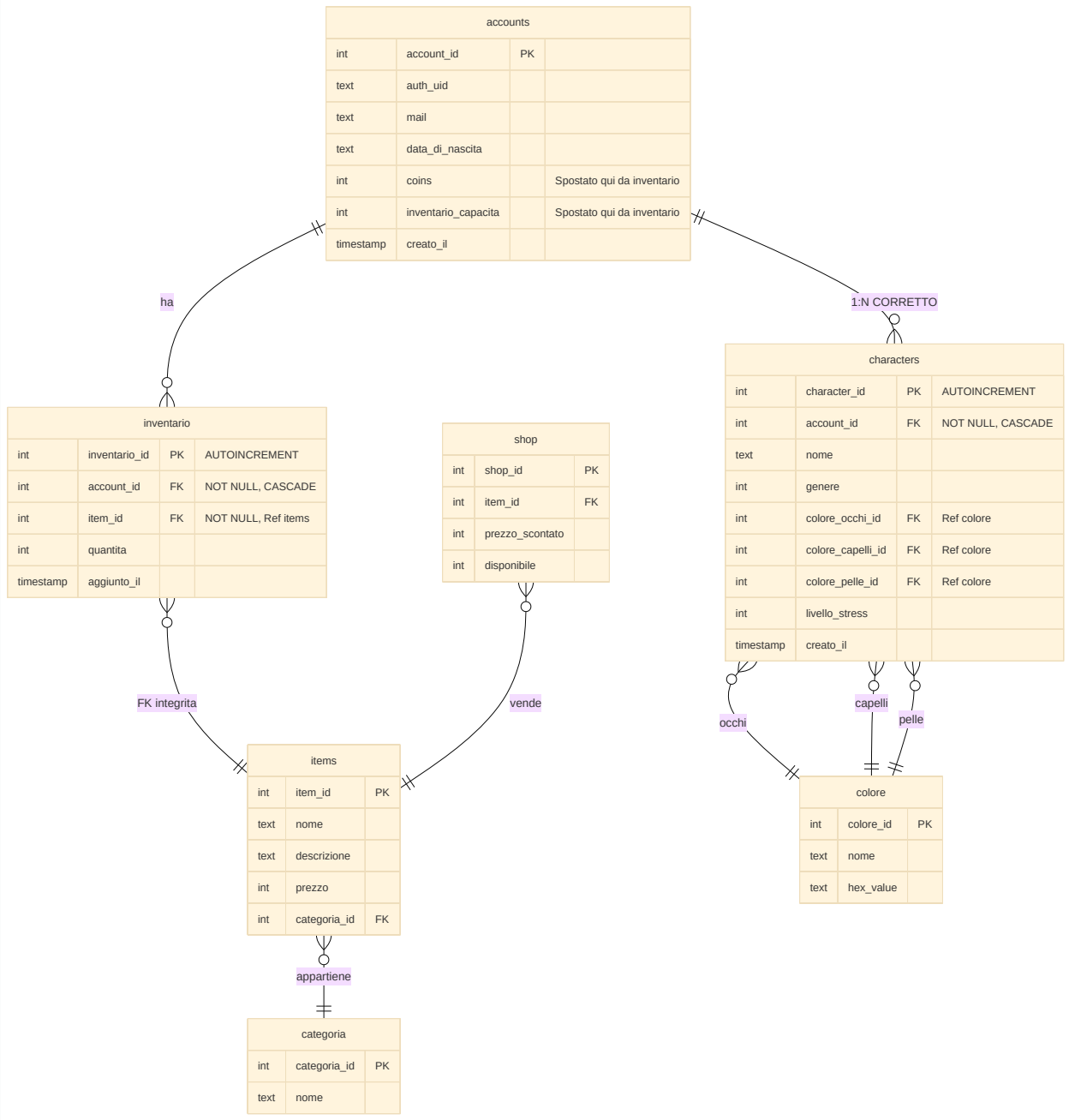


Figura 8 — Schema ER proposto (corretto)

Obiettivo: Ridisegnare lo schema del database per supportare multipli personaggi e un inventario coerente.

Passo 1 — Nuova tabella characters:

Prima (schema errato):

```sql
-- PRIMA: account_id come PRIMARY KEY
-- Un solo personaggio per account!
CREATE TABLE IF NOT EXISTS characters (
    account_id INTEGER PRIMARY KEY,
    nome TEXT DEFAULT '',
    -- ... altri campi ...
);
```

Dopo (schema corretto):

```sql
-- DOPO: character_id come PRIMARY KEY
-- Piu' personaggi per account!
CREATE TABLE IF NOT EXISTS characters (
    character_id INTEGER PRIMARY KEY AUTOINCREMENT,
    -- character_id: numero univoco auto-incrementante per ogni personaggio
    account_id INTEGER NOT NULL REFERENCES accounts(account_id) ON DELETE CASCADE,
    -- account_id: a quale account appartiene questo personaggio
    -- REFERENCES: e' una FOREIGN KEY che punta alla tabella accounts
    -- ON DELETE CASCADE: se l'account viene eliminato, anche i personaggi vengono elimin
    nome TEXT DEFAULT '',
    -- nome: il nome del personaggio
    genere INTEGER DEFAULT 0,
    -- genere: 0 = maschio, 1 = femmina (o altri valori per altri generi)
    colore_occhi_id INTEGER DEFAULT NULL REFERENCES colore(colore_id),
    -- colore_occhi_id: FOREIGN KEY verso la tabella dei colori
    colore_capelli_id INTEGER DEFAULT NULL REFERENCES colore(colore_id),
    colore_pelle_id INTEGER DEFAULT NULL REFERENCES colore(colore_id),
    livello_stress INTEGER DEFAULT 0,
    creato_il TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    -- creato_il: data e ora di creazione, automaticamente impostata
    UNIQUE(account_id, nome)
    -- UNIQUE: la combinazione account + nome deve essere unica
    -- (non puoi avere due personaggi con lo stesso nome nello stesso account)
);
```

Passo 2 — Ristrutturare inventario:

Prima (schema errato):

```sql
-- PRIMA: coins e capacita sono per ogni oggetto (non ha senso!)
CREATE TABLE IF NOT EXISTS inventario (
    account_id INTEGER,
    item_id INTEGER,
    coins INTEGER DEFAULT 0,     -- perche' le monete sono qui?!
    capacita INTEGER DEFAULT 50, -- perche' la capacita' e' qui?!
    -- ...
);
```

Dopo (schema corretto):

```sql
-- DOPO: coins e capacita vanno nell'account, non nell'inventario
-- Passo A: Aggiungiamo i campi alla tabella accounts
ALTER TABLE accounts ADD COLUMN coins INTEGER DEFAULT 0;
-- coins: le monete dell'utente (una sola volta per account)
ALTER TABLE accounts ADD COLUMN inventario_capacita INTEGER DEFAULT 50;
-- inventario_capacita: quanti oggetti puo' avere l'utente

-- Passo B: Ristrutturiamo la tabella inventario
CREATE TABLE IF NOT EXISTS inventario (
    inventario_id INTEGER PRIMARY KEY AUTOINCREMENT,
    -- inventario_id: identificatore unico per ogni riga
    account_id INTEGER NOT NULL REFERENCES accounts(account_id) ON DELETE CASCADE,
    -- account_id: a quale account appartiene questo oggetto
    item_id INTEGER NOT NULL REFERENCES items(item_id),
    -- item_id: FOREIGN KEY verso la tabella items
    -- Ora c'e' integrita' referenziale: non si possono inserire oggetti inesistenti
    quantita INTEGER DEFAULT 1,
    -- quantita: quanti di questo oggetto ha l'utente
    aggiunto_il TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    UNIQUE(account_id, item_id)
    -- UNIQUE: un utente non puo' avere due righe per lo stesso oggetto
    -- (aumenta la quantita' invece di duplicare la riga)
);
```

Impatto: Questa modifica richiede l'aggiornamento di tutte le funzioni che usano `account_id` come lookup per i personaggi, e tutte le funzioni che accedono a `coins` e `capacita` nell'inventario.

Fase 2 — Gestione Memoria e Lifecycle (ALTO)

Concetto: Il Ciclo di Vita dei Nodi in Godot

Ogni nodo in Godot ha un "ciclo di vita": nasce (`_ready()`), vive (`_process()` , `_input()`), e muore (`_exit_tree()`). Il problema piu' comune nei progetti Godot — e certamente il piu' comune in questo progetto — e' dimenticare la fase di "morte": non pulire quando un nodo viene distrutto.

Pensate a una festa. Se 10 persone arrivano a una festa (connettono segnali) e se ne vanno senza pulire (senza disconnettere), i piatti sporchi (connessioni orfane) si accumulano. Dopo 10 feste (10 cambi di scena), la casa e' sommersa da piatti sporchi (memory leak).

2.1 Aggiungere `_exit_tree()` a tutti gli script

Script da correggere (12 in totale): panel_manager, shop_panel, deco_panel, settings_panel, room_base, decoration_system, room_grid, main_menu, menu_character, main, music_panel (parziale), character_controller.

Per ogni script, il processo e' lo stesso:

Passo 1: Aprite il file e trovate la funzione `_ready()`. Elencate tutti i segnali connessi.

Passo 2: Alla fine del file, aggiungete (o completate) la funzione `_exit_tree()` che disconnette tutti quei segnali.

Template generico:

```
# Questa funzione viene chiamata quando il nodo sta per essere rimosso
# E' FONDAMENTALE per prevenire memory leak e crash
func _exit_tree() -> void:
    # 1. Disconnettere tutti i segnali SignalBus
    # Per OGNI segnale connesso in _ready(), aggiungiamo una disconnessione
    # Il check "is_connected" evita errori se il segnale era gia' disconnesso
    if SignalBus.room_changed.is_connected(_on_room_changed):
        SignalBus.room_changed.disconnect(_on_room_changed)
    if SignalBus.decoration_placed.is_connected(_on_decoration_placed):
        SignalBus.decoration_placed.disconnect(_on_decoration_placed)
    # ... ripetere per ogni segnale connesso in _ready()

    # 2. Fermare timer attivi
    # Un timer non fermato continua a scattare dopo la distruzione del nodo
    if _timer and not _timer.is_stopped():
        _timer.stop()

    # 3. Killare tween attivi
    # Un tween non killato continua la sua animazione su un nodo distrutto
    if _tween and _tween.is_running():
        _tween.kill()
```

Come verificare: Dopo aver applicato la correzione, aprite e chiudete il pannello (o cambiate scena) 100 volte. Se il Profiler di Godot non mostra aumento costante di memoria, la correzione funziona.

2.2 Correggere music_panel.gd FileDialog

Obiettivo: Creare il FileDialog una sola volta e riutilizzarlo, invece di crearne uno nuovo ad ogni click.

Prima (codice problematico):

```
# PRIMA: ogni click crea un NUOVO FileDialog
# che non viene MAI distrutto = memory leak!
func _on_import_pressed() -> void:
    var dialog = FileDialog.new()  # nuovo oggetto ogni volta!
    dialog.file_mode = FileDialog.FILE_MODE_OPEN_FILE
    # ... configurazione ...
    add_child(dialog)  # aggiunto alla scena ma mai rimosso!
    dialog.popup_centered(Vector2i(600, 400))
```

Dopo (codice corretto):

```
# DOPO: creiamo il FileDialog UNA sola volta e lo riutilizziamo
# Variabile membro della classe — persiste per tutta la vita del pannello
var _file_dialog: FileDialog = null

func _on_import_pressed() -> void:
    # Su piattaforma web il FileDialog non e' supportato
    if OS.has_feature("web"):
        return

    # Creiamo il dialog solo la prima volta
    if _file_dialog == null:
        _file_dialog = FileDialog.new()
        _file_dialog.file_mode = FileDialog.FILE_MODE_OPEN_FILE
        _file_dialog.access = FileDialog.ACCESS_FILESYSTEM
        # Accettiamo solo file audio MP3 e WAV
        _file_dialog.filters = PackedStringArray(["*.mp3", "*.wav"])
        # Connettiamo il segnale che ci dice quale file e' stato scelto
        _file_dialog.file_selected.connect(_on_file_selected)
        # Aggiungiamo il dialog alla scena
        add_child(_file_dialog)

    # Mostriamo il dialog (che sia nuovo o gia' esistente)
    _file_dialog.popup_centered(Vector2i(600, 400))

# Pulizia: distruggiamo il dialog quando il pannello viene rimosso
func _exit_tree() -> void:
    # Verifichiamo che il dialog esista e sia ancora valido
    if _file_dialog and is_instance_valid(_file_dialog):
        _file_dialog.queue_free()  # lo eliminiamo in modo sicuro
    # ... altre disconnessioni di segnali ...
```

2.3 Correggere room_base.gd race condition

Obiettivo: Eliminare la race condition nel cambio di personaggio.

Prima (codice problematico):

```
# PRIMA: il vecchio personaggio viene "schedulato" per l'eliminazione
# ma il nuovo viene aggiunto immediatamente
# Per un breve momento, entrambi esistono = conflitto!
func _swap_character(new_scene_path: String) -> void:
    old_character.queue_free()  # sara' eliminato a fine frame
    var new_char = load(new_scene_path).instantiate()
    add_child(new_char)  # aggiunto ORA, ma il vecchio e' ancora qui!
```

Dopo (codice corretto):

```
# DOPO: usiamo call_deferred per aggiungere il nuovo personaggio
# solo DOPO che il vecchio e' stato effettivamente eliminato
func _swap_character(new_scene_path: String) -> void:
    old_character.queue_free()  # sara' eliminato a fine frame
    var new_char = load(new_scene_path).instantiate()
    # call_deferred ritarda l'aggiunta alla fine del frame
    # A quel punto, queue_free avra' gia' eliminato il vecchio
    call_deferred("add_child", new_char)
```

2.4 Correggere audio_manager.gd ambience

Obiettivo: Garantire che i player audio ambientali vengano correttamente distrutti.

```
# Per ogni player ambientale, prima verifichiamo che sia ancora valido
# poi lo rimuoviamo dal dizionario e lo distruggiamo
func _stop_ambience(key: String) -> void:
    if _ambience_players.has(key):
        var player = _ambience_players[key]
        # Rimuoviamo dal dizionario PRIMA di distruggere
        # Cosi' nessun altro codice puo' usare un player in via di distruzione
        _ambience_players.erase(key)
        # Verifichiamo che il player sia ancora valido
        # (potrebbe essere gia' stato distrutto da un altro evento)
        if is_instance_valid(player):
            player.stop()  # fermiamo la riproduzione
            player.queue_free()  # eliminiamo il nodo
```

2.5 Aggiungere bounds check audio tracks

Obiettivo: Prevenire il crash quando la lista delle tracce e' vuota.

```
# Prima di accedere alla lista delle tracce, verifichiamo che non sia vuota
# e che l'indice sia valido
func _play_current_track() -> void:
    # Controllo 1: la lista non deve essere vuota
    if tracks.is_empty():
        push_warning("AudioManager: nessuna traccia disponibile")
        return  # usciamo senza fare nulla

    # Controllo 2: l'indice deve essere nei limiti validi
    # clampi forza il valore tra 0 e l'ultimo indice valido
    current_track_index = clampi(current_track_index, 0, tracks.size() - 1)

    # Ora possiamo accedere in sicurezza
    var track = tracks[current_track_index]
    # ... riproduzione traccia ...
```

Fase 3 — Gestione Errori e Validazione (MEDIO)

Concetto: Programmazione Difensiva

La **programmazione difensiva** e' una filosofia di sviluppo in cui il codice non si fida di nessuno: non si fida dei dati in ingresso, non si fida che le risorse esistano, non si fida che le operazioni vadano a buon fine. Ogni operazione viene verificata, ogni valore viene validato.

Pensate a un buttafuori all'ingresso di un locale: controlla il documento di tutti, anche di chi sembra ovviamente maggiorenne. Nel software, questo approccio previene la maggior parte dei crash e dei comportamenti anomali.

3.1 Null check su tutti i riferimenti manager

Obiettivo: Ogni script che accede a un autoload deve verificare che esista.

Prima (codice senza protezione):

```
# PRIMA: se GameManager e' null, CRASH!
func _ready() -> void:
    var room = GameManager.current_room  # crash se GameManager e' null
```

Dopo (codice con protezione):

```
# DOPO: verifichiamo che GameManager esista
func _ready() -> void:
    if GameManager == null:
        # Logghiamo un errore chiaro che spiega il problema
        push_error("GameManager non inizializzato — verificare ordine autoload")
        return  # usciamo dalla funzione senza crashare

    var room = GameManager.current_room  # ora e' sicuro
```

3.2 Validazione texture load

Obiettivo: Ogni caricamento di texture deve essere verificato.

Prima:

```
# PRIMA: se il percorso e' sbagliato, tex e' null e la riga dopo crasha
var tex = load(path) as Texture2D
sprite.texture = tex  # CRASH se tex e' null!
```

Dopo:

```
# DOPO: verifichiamo che la texture sia stata caricata correttamente
var tex := load(path) as Texture2D
if tex == null:
    # La texture non e' stata trovata o non e' del tipo giusto
    push_error("Texture non trovata: %s" % path)
    return  # usciamo senza crashare

# Ora possiamo usare la texture in sicurezza
sprite.texture = tex
```

3.3 Validazione struttura dati salvataggio

Obiettivo: Aggiungere una funzione che verifica l'integrita' dei dati di salvataggio prima di usarli.

```
# Questa funzione verifica che i dati di salvataggio abbiano la struttura corretta
# Ritorna true se i dati sono validi, false altrimenti
func _validate_save_data(data: Dictionary) -> bool:
    # Verifichiamo che il dizionario contenga le chiavi fondamentali
    var required_keys: Array = ["version", "character", "inventory", "settings"]

    for key in required_keys:
        if not data.has(key):
            # Manca una chiave fondamentale
            AppLogger.error("SaveManager", "Dati salvataggio incompleti", {"chiave_mancan
            return false  # dati non validi

    # Verifichiamo che la versione sia un numero valido
    if not str(data["version"]).is_valid_int():
        AppLogger.error("SaveManager", "Versione salvataggio non valida", {"versione": da
        return false

    # Se arriviamo qui, i dati hanno la struttura base corretta
    return true
```

3.4 Version comparison safety

Obiettivo: Rendere la comparazione delle versioni robusta contro formati non standard.

Prima (codice fragile):

```
# PRIMA: int() crasha se la stringa non e' un numero puro
func _compare_versions(a: String, b: String) -> int:
    # Se a = "1.0.0-beta", int("beta") causa errore!
    return int(a) - int(b)
```

Dopo (codice robusto):

```
# DOPO: gestiamo versioni in qualsiasi formato
func _compare_versions(a: String, b: String) -> int:
    # Dividiamo le versioni per punti: "1.2.3" -> ["1", "2", "3"]
    var parts_a := a.split(".")
    var parts_b := b.split(".")

    # Confrontiamo fino a 3 componenti (major.minor.patch)
    for i: int in 3:
        # Per ogni componente, convertiamo a intero solo se e' un numero valido
        # Altrimenti usiamo 0 come valore di default
        var va: int = int(parts_a[i]) if i < parts_a.size() and parts_a[i].is_valid_int()
        var vb: int = int(parts_b[i]) if i < parts_b.size() and parts_b[i].is_valid_int()

        # Se i componenti sono diversi, ritorniamo la differenza
        if va != vb:
            return va - vb  # positivo se a > b, negativo se a < b

    # Se tutti i componenti sono uguali, le versioni sono identiche
    return 0
```

Fase 4 — Allineamento Architetturale (ARCHITETTURALE)

Concetto: Comunicazione Tramite Segnali, Non Tramite Chiamate Dirette

L'architettura signal-driven funziona cosi': i componenti non si chiamano direttamente, ma comunicano tramite segnali. Questo e' fondamentale perche':

1. **Disaccoppiamento**: ogni componente puo' essere modificato senza toccare gli altri
2. **Testabilita'**: ogni componente puo' essere testato in isolamento
3. **Flessibilita'**: e' facile aggiungere nuovi "ascoltatori" senza modificare chi emette il segnale

Immaginate un'orchestra. Se ogni musicista dovesse dire personalmente al direttore "ho finito il mio assolo", sarebbe caotico. Invece, seguono tutti la partitura (i segnali): quando e' il momento, suonano; quando finiscono, il prossimo musicista sa che tocca a lui.

4.1 Nuovo segnale per settings update

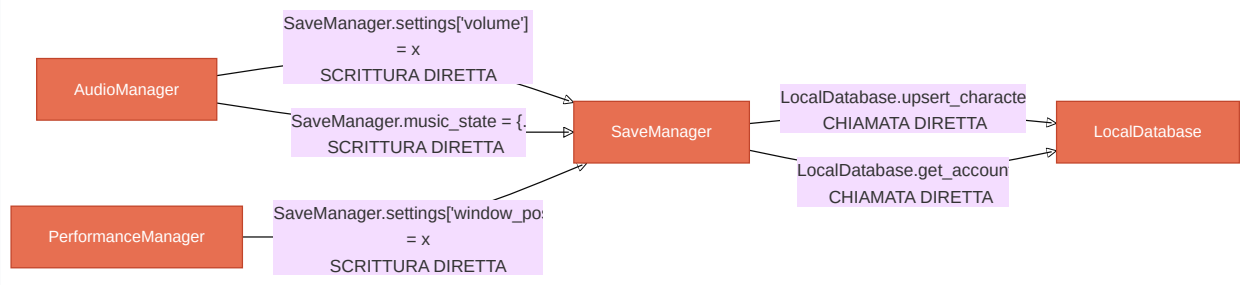


Figura 6 — Flusso comunicazione PRIMA (coupling diretto)

Obiettivo: Sostituire le scritture dirette in `SaveManager.settings` con segnali.

Passo 1 — Aggiungere il segnale in signal_bus.gd:

```
# In signal_bus.gd
# Questo segnale viene emesso quando un'impostazione cambia
# key: il nome dell'impostazione (es. "volume_music")
# value: il nuovo valore (puo' essere qualsiasi tipo grazie a Variant)
signal settings_updated(key: String, value: Variant)
```

Passo 2 — Modificare i componenti che scrivevano direttamente:

Prima (AudioManager, codice accoppiato):

```
# PRIMA: l'AudioManager modifica direttamente i dati del SaveManager
# Questo crea una dipendenza diretta = coupling alto
func _on_volume_changed(bus_name: String, value: float) -> void:
    SaveManager.settings["volume_" + bus_name] = value  # scrittura diretta!
```

Dopo (AudioManager, codice disaccoppiato):

```
# DOPO: l'AudioManager emette un segnale
# Non gli importa chi lo ascolta o cosa fa con il valore
func _on_volume_changed(bus_name: String, value: float) -> void:
    # Emettiamo il segnale: "un'impostazione e' cambiata"
    SignalBus.settings_updated.emit("volume_" + bus_name, value)
```

Passo 3 — SaveManager ascolta il segnale:

```
# In save_manager.gd _ready()
# Ci connettiamo al segnale per ricevere gli aggiornamenti
SignalBus.settings_updated.connect(_on_settings_updated)

# La funzione che riceve gli aggiornamenti
func _on_settings_updated(key: String, value: Variant) -> void:
    settings[key] = value  # aggiorniamo il nostro dizionario
    _mark_dirty()  # segniamo che ci sono modifiche non salvate
```

Ripetete lo stesso pattern per PerformanceManager e settings_panel.

4.2 Nuovo segnale per music state

```
# In signal_bus.gd
# Questo segnale viene emesso quando lo stato musicale cambia
signal music_state_updated(state: Dictionary)
```

Prima (AudioManager scrive direttamente):

```
# PRIMA
func _sync_music_state() -> void:
    SaveManager.music_state = _get_current_state()  # scrittura diretta!
```

Dopo (AudioManager emette segnale):

```
# DOPO
func _sync_music_state() -> void:
    # Emettiamo lo stato, SaveManager lo raccogliera'
    SignalBus.music_state_updated.emit(_get_current_state())
```

4.3 SaveManager verso LocalDatabase via segnale

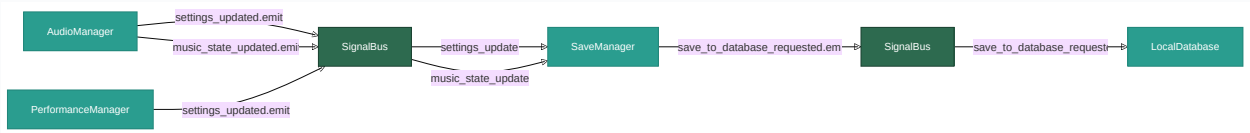


Figura 7 — Flusso comunicazione DOPO (via SignalBus)

```
# In signal_bus.gd
# Questo segnale richiede il salvataggio dei dati sul database
signal save_to_database_requested(data: Dictionary)
```

Prima:

```
# PRIMA: SaveManager chiama direttamente LocalDatabase
func _save_to_sqlite() -> void:
    LocalDatabase.upsert_character(char_data)  # chiamata diretta
```

Dopo:

```
# DOPO: SaveManager emette un segnale
func _save_to_sqlite() -> void:
    SignalBus.save_to_database_requested.emit({"character": char_data, "inventory": inv_d

# In local_database.gd _ready()
SignalBus.save_to_database_requested.connect(_on_save_requested)
```

Fase 5 — Copertura Test (TEST)

Concetto: Perche' i Test Sono Indispensabili

Immaginate di costruire un ponte. Dopo averlo costruito, lo testereste prima di aprirlo al traffico, giusto? Non lascereste passare i camion sperando che regga. I test del software funzionano allo stesso modo: verificano che ogni componente faccia esattamente quello che deve fare, PRIMA che il codice arrivi agli utenti.

L'obiettivo e' portare la copertura dal 15-20% attuale al 50%+.

5.1 Test AudioManager

Creare il file `tests/unit/test_audio_manager.gd`:

```
# test_audio_manager.gd
# Test per il gestore audio del gioco
# Verifichiamo che la musica, i suoni e le playlist funzionino correttamente

extends GdUnitTestSuite  # la classe base per i test GdUnit4

# Test: il bounds check impedisce crash su lista vuota
func test_empty_tracks_no_crash() -> void:
    # Simuliamo una lista di tracce vuota
    AudioManager.tracks = []
    # Proviamo a riprodurre: non deve crashare
    AudioManager.play_current()
    # Se arriviamo qui senza crash, il test e' passato
    assert_true(true)

# Test: l'indice della traccia resta nei limiti validi
func test_track_index_bounds() -> void:
    AudioManager.tracks = ["track1.mp3", "track2.mp3"]
    AudioManager.current_track_index = 999  # indice fuori range
    AudioManager._ensure_valid_index()
    # L'indice deve essere stato corretto al massimo valido (1)
    assert_eq(AudioManager.current_track_index, 1)

# Test: le modalita' playlist sono tutte gestite
func test_playlist_modes() -> void:
    var valid_modes = ["sequential", "shuffle", "repeat"]
    for mode in valid_modes:
        AudioManager.playlist_mode = mode
        # Nessun errore o comportamento inatteso
        assert_true(AudioManager.playlist_mode in valid_modes)
```

5.2 Test LocalDatabase

Creare il file `tests/unit/test_local_database.gd`:

```
# test_local_database.gd
# Test per il database locale
# Verifichiamo che le operazioni CRUD funzionino e lo schema sia corretto

extends GdUnitTestSuite

# Test: inserimento e lettura di un personaggio
func test_upsert_and_read_character() -> void:
    var char_data: Dictionary = {
        "nome": "Test Hero",
        "genere": 0,
        "livello_stress": 3
    }
    # Inseriamo il personaggio
    var result = LocalDatabase.upsert_character(char_data)
    assert_true(result)  # l'inserimento deve riuscire

    # Leggiamo il personaggio appena inserito
    var characters = LocalDatabase.get_characters(1)
    assert_false(characters.is_empty())  # deve esserci almeno un risultato
    assert_eq(characters[0]["nome"], "Test Hero")  # il nome deve corrispondere

# Test: due personaggi per lo stesso account (dopo fix C3)
func test_multiple_characters_per_account() -> void:
    LocalDatabase.upsert_character({"nome": "Eroe 1", "genere": 0})
    LocalDatabase.upsert_character({"nome": "Eroe 2", "genere": 1})
    var characters = LocalDatabase.get_characters(1)
    # Devono esserci 2 personaggi
    assert_eq(characters.size(), 2)

# Test: foreign key impedisce inserimento di item inesistenti
func test_foreign_key_integrity() -> void:
    # Tentiamo di inserire un oggetto con item_id inesistente
    var result = LocalDatabase.add_inventory_item(1, 99999, 0, 50)
    # L'inserimento deve fallire per violazione di foreign key
    assert_false(result)
```

5.3 Test Room Logic

Creare il file `tests/unit/test_room_base.gd`:

```
# test_room_base.gd
# Test per la logica delle stanze
# Verifichiamo che decorazioni e personaggi funzionino correttamente

extends GdUnitTestSuite

# Test: piazzamento di una decorazione
func test_decoration_spawn() -> void:
    # Simuliamo il piazzamento di una decorazione
    var room = auto_free(preload("res://scenes/main.tscn").instantiate())
    add_child(room)
    # Emettiamo il segnale di piazzamento
    SignalBus.decoration_placed.emit("lamp_01", Vector2(100, 200))
    # Verifichiamo che la decorazione sia stata aggiunta
    # (il numero di figli deve essere aumentato)
    assert_true(room.get_child_count() > 0)

# Test: cambio personaggio senza crash
func test_character_swap_no_crash() -> void:
    var room = auto_free(preload("res://scenes/main.tscn").instantiate())
    add_child(room)
    # Cambiamo personaggio rapidamente 5 volte
    for i in 5:
        SignalBus.character_changed.emit("female_yellow_shirt")
        await get_tree().process_frame  # aspettiamo un frame
    # Se arriviamo qui senza crash, il test e' passato
    assert_true(true)
```

5.4 Test UI Panel Lifecycle

Creare il file `tests/unit/test_panel_manager.gd`:

```
# test_panel_manager.gd
# Test per il gestore dei pannelli UI
# Verifichiamo che apertura, chiusura e pulizia funzionino

extends GdUnitTestSuite

# Test: aprire e chiudere un pannello non causa memory leak
func test_open_close_no_leak() -> void:
    var initial_objects = Performance.get_monitor(Performance.OBJECT_COUNT)
    # Apriamo e chiudiamo il pannello 10 volte
    for i in 10:
        PanelManager.open_panel("shop")
        await get_tree().process_frame
        PanelManager.close_panel("shop")
        await get_tree().process_frame
    var final_objects = Performance.get_monitor(Performance.OBJECT_COUNT)
    # Il numero di oggetti non deve crescere significativamente
    assert_true(final_objects - initial_objects < 5)

# Test: un solo pannello alla volta
func test_mutual_exclusion() -> void:
    PanelManager.open_panel("shop")
    PanelManager.open_panel("music")
    # Solo il secondo pannello deve essere aperto
    assert_false(PanelManager.is_open("shop"))
    assert_true(PanelManager.is_open("music"))
```

5.5 Espandere test ShopPanel

Aggiungere i seguenti test al file `tests/unit/test_shop_panel.gd` esistente:

```
# Nuovi test da aggiungere a test_shop_panel.gd

# Test: il filtro per categoria funziona
func test_category_filtering() -> void:
    # Selezioniamo la categoria "cucina"
    shop_panel._on_category_selected("cucina")
    # Verifichiamo che vengano mostrati solo oggetti della cucina
    for item in shop_panel._displayed_items:
        assert_eq(item["category"], "cucina")

# Test: il conteggio degli oggetti e' corretto
func test_item_count_matches_catalog() -> void:
    var catalog = GameManager.get_decoration_catalog()
    var total_displayed = shop_panel._displayed_items.size()
    # Il numero di oggetti mostrati deve corrispondere al catalogo
    assert_eq(total_displayed, catalog.size())

# Test: i dati del drag sono integri
func test_drag_data_integrity() -> void:
    var drag_data = shop_panel._get_drag_data_for_item("lamp_01")
    # I dati di drag devono contenere le informazioni necessarie
    assert_true(drag_data.has("item_id"))
    assert_true(drag_data.has("sprite_path"))
    assert_true(drag_data.has("preview"))
```

12. Istruzioni Dettagliate per Correzione

Questa sezione fornisce istruzioni passo per passo, in stile tutorial, per le correzioni piu' importanti. Per ogni correzione: 1. Vi spieghiamo cosa vedrete quando aprite il file 2. Vi spieghiamo il codice esistente riga per riga 3. Vi spieghiamo la correzione riga per riga 4. Vi spieghiamo come verificare che funzioni

---

C1 — Inventario non salvato su SQLite

File da modificare: `scripts/autoload/save_manager.gd`

Cosa vedrete quando aprite il file: Il SaveManager e' un file di circa 290 righe che gestisce il salvataggio e il caricamento dei dati. Cercate la funzione `_save_to_sqlite()` — si trova intorno alla riga 115.

Il codice esistente (riga ~115-120):

```
func _save_to_sqlite() -> void:
    # Questa funzione salva i dati sul database SQLite
    # come backup del file JSON principale

    # Salva i dati del personaggio
    LocalDatabase.upsert_character(character_data)
    # ... MA L'INVENTARIO NON VIENE SALVATO!
    # Qui manca completamente il salvataggio degli oggetti
```

La correzione: Aggiungete il seguente blocco subito dopo `upsert_character()`:

```
func _save_to_sqlite() -> void:
    # Salva i dati del personaggio (codice esistente)
    LocalDatabase.upsert_character(character_data)

    # === INIZIO NUOVA CORREZIONE ===
    # Salviamo anche l'inventario, che prima veniva ignorato
    # Controlliamo che ci siano effettivamente oggetti da salvare
    if not inventory_data.get("items", []).is_empty():
        # Iteriamo su ogni oggetto nell'inventario
        for item: Dictionary in inventory_data["items"]:
            # Aggiungiamo ogni oggetto al database
            LocalDatabase.add_inventory_item(
                1,  # ID dell'account locale (sempre 1 per single-player)
                item.get("item_id", 0),  # ID dell'oggetto, 0 se non specificato
                inventory_data.get("coins", 0),  # Monete dell'utente
                inventory_data.get("capacity", 50)  # Capacita' massima inventario
            )
    # === FINE NUOVA CORREZIONE ===
```

Prerequisito: Questa correzione funziona correttamente solo DOPO aver corretto lo schema dell'inventario (C4). Se applicate C1 senza C4, i dati verranno salvati nella struttura sbagliata.

Come verificare: 1. Avviate il gioco e acquistate alcuni oggetti 2. Salvate il gioco (manualmente o aspettando l'auto-save) 3. Aprite il database SQLite con uno strumento come DB Browser for SQLite 4. Eseguite la query: `SELECT * FROM inventario` 5. Dovreste vedere gli oggetti che avete acquistato

---

C2 — Backup senza error checking

File da modificare: `scripts/autoload/save_manager.gd`

Cosa vedrete: Nella funzione `save_game()`, cercate la riga dove viene copiato il file di backup (intorno alla riga 92-93). Vedrete qualcosa come `DirAccess.copy_absolute(...)` senza nessun controllo del risultato.

Il codice esistente (riga ~92-93):

```
# Il codice originale copia il file senza controllare se e' andato bene
DirAccess.copy_absolute(SAVE_PATH, BACKUP_PATH)
# Se la copia fallisce (disco pieno, permessi insufficienti),
# nessuno lo sapra' mai!
```

La correzione:

```
# Apriamo la directory dove si trovano i file di salvataggio
var dir := DirAccess.open("user://")
if dir:
    # Tentiamo di copiare il file corrente come backup
    var err := dir.copy(SAVE_PATH, BACKUP_PATH)
    if err != OK:
        # La copia e' fallita! Logghiamo l'errore con i dettagli
        # In questo modo lo sviluppatore puo' diagnosticare il problema
        AppLogger.error("SaveManager", "Backup fallito", {"errore": err})
        # Nota: NON interrompiamo il salvataggio — proviamo comunque
        # a salvare il file principale. Meglio un salvataggio senza backup
        # che nessun salvataggio.
else:
    # Non riusciamo nemmeno ad accedere alla directory user://
    # Questo e' un problema serio: potrebbe indicare un problema
    # con i permessi del filesystem
    AppLogger.error("SaveManager", "Directory user:// non accessibile")
```

Come verificare: 1. Simulate un disco pieno (rinominate la directory di backup con permessi sola-lettura) 2. Salvate il gioco 3. Controllate i log: dovreste vedere il messaggio di errore 4. Verificate che il salvataggio principale sia comunque avvenuto

---

C3 — Characters PRIMARY KEY impedisce multipli personaggi

File da modificare: `scripts/autoload/local_database.gd`

Cosa vedrete: Nella funzione `_create_tables()` (intorno alla riga 101-102), troverete la definizione SQL della tabella `characters`.

Il codice esistente:

```
-- La tabella characters originale
-- account_id come PRIMARY KEY = UN SOLO personaggio per account
CREATE TABLE IF NOT EXISTS characters (
    account_id INTEGER PRIMARY KEY,  -- ERRORE: dovrebbe essere character_id!
    nome TEXT DEFAULT '',
    genere INTEGER DEFAULT 0,
    -- ... altri campi ...
);
```

La correzione:

```sql
-- La tabella characters corretta
-- character_id come PRIMARY KEY = MULTIPLI personaggi per account
CREATE TABLE IF NOT EXISTS characters (
    character_id INTEGER PRIMARY KEY AUTOINCREMENT,
    -- character_id: ogni personaggio ha il suo ID univoco
    -- AUTOINCREMENT: il database assegna automaticamente il prossimo numero
    account_id INTEGER NOT NULL REFERENCES accounts(account_id) ON DELETE CASCADE,
    -- account_id: a quale account appartiene il personaggio
    -- NOT NULL: obbligatorio (ogni personaggio DEVE appartenere a un account)
    -- REFERENCES: foreign key verso la tabella accounts
    -- ON DELETE CASCADE: se l'account viene eliminato, tutti i suoi personaggi
    --                    vengono eliminati automaticamente
    nome TEXT DEFAULT '',
    genere INTEGER DEFAULT 0,
    colore_occhi_id INTEGER DEFAULT NULL REFERENCES colore(colore_id),
    colore_capelli_id INTEGER DEFAULT NULL REFERENCES colore(colore_id),
    colore_pelle_id INTEGER DEFAULT NULL REFERENCES colore(colore_id),
    livello_stress INTEGER DEFAULT 0,
    creato_il TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    UNIQUE(account_id, nome)
    -- UNIQUE su (account_id, nome): nello stesso account,
    -- non possono esserci due personaggi con lo stesso nome
);
```

Impatto: Dopo questa modifica, dovrete aggiornare tutte le funzioni che cercano personaggi per `account_id`: - `get_character(account_id)` deve diventare `get_characters(account_id)` (plurale) e ritornare un array - `upsert_character()` deve includere `account_id` come parametro obbligatorio - Le funzioni di caricamento nel SaveManager devono gestire il fatto che un account possa avere piu' personaggi

Come verificare: 1. Create un personaggio "Eroe A" per l'account 1 2. Create un personaggio "Eroe B" per lo stesso account 1 3. Eseguite: `SELECT * FROM characters WHERE account_id = 1` 4. Dovreste vedere 2 righe, non 1

C4 — Inventory schema confuso

File da modificare: `scripts/autoload/local_database.gd`

Cosa vedrete: La definizione della tabella `inventario` con campi `coins` e `capacita` per ogni riga.

La correzione ha due parti:

Parte A — Aggiungere coins e capacita alla tabella accounts:

```sql
-- Aggiungiamo i campi che riguardano l'ACCOUNT (non il singolo oggetto)
ALTER TABLE accounts ADD COLUMN coins INTEGER DEFAULT 0;
-- coins: le monete dell'utente — un valore per account
ALTER TABLE accounts ADD COLUMN inventario_capacita INTEGER DEFAULT 50;
-- inventario_capacita: quanti oggetti puo' avere — un valore per account
```

Parte B — Ristrutturare la tabella inventario:

```sql
-- La nuova tabella inventario: semplice relazione account-oggetto
CREATE TABLE IF NOT EXISTS inventario (
    inventario_id INTEGER PRIMARY KEY AUTOINCREMENT,
    -- ID univoco per ogni riga dell'inventario
    account_id INTEGER NOT NULL REFERENCES accounts(account_id) ON DELETE CASCADE,
    -- A quale account appartiene questo oggetto
    item_id INTEGER NOT NULL REFERENCES items(item_id),
    -- Quale oggetto e': FOREIGN KEY verso la tabella items
    -- Questo garantisce integrita' referenziale:
    -- non si possono inserire oggetti che non esistono nel catalogo
    quantita INTEGER DEFAULT 1,
    -- Quanti di questo oggetto ha l'utente
    aggiunto_il TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    -- Quando l'oggetto e' stato aggiunto all'inventario
    UNIQUE(account_id, item_id)
    -- Un utente non puo' avere due righe per lo stesso oggetto
    -- Se compra un oggetto che ha gia', si incrementa la quantita'
);
```

Come verificare: 1. Acquistate un oggetto nel negozio 2. Verificate che `coins` nella tabella `accounts` sia diminuito 3. Verificate che l'oggetto sia presente nella tabella `inventario` 4. Tentate di inserire un `item_id` inesistente: deve fallire

---

C5 — Array mismatch window_background.gd

File da modificare: `scripts/rooms/window_background.gd`

Cosa vedrete: La funzione `_build_layers()` che carica i layer dello sfondo uno per uno.

Le istruzioni dettagliate per questa correzione sono gia' state fornite nella Fase 1, Sezione 1.2 del Piano di Stabilizzazione. Il principio chiave e': quando un layer fallisce, saltare COMPLETAMENTE quel layer (sia la sprite che il factor), cosi' i due array restano sempre allineati.

Come verificare: 1. Rinominate temporaneamente uno dei file di layer dello sfondo (cosi' non viene trovato) 2. Avviate il gioco 3. Il gioco deve funzionare senza crash, con un layer mancante 4. Nei log deve apparire un avviso che il layer non e' stato trovato 5. Ripristinate il nome del file originale

C6 — Typo percorso sprite characters.json

File da modificare: `data/characters.json`

Cosa vedrete: Nel file JSON, cercate la sezione `male_old`. Nelle animazioni di camminata, troverete un percorso sprite con un errore di battitura.

La correzione: Cercate `male_walk_down_side_sxt.png` e sostituitelo con `male_walk_down_side_sx.png` (rimuovete la "t" in eccesso).

Come verificare: 1. Avviate il gioco 2. Selezionate il personaggio `male_old` 3. Fate camminare il personaggio verso il basso e di lato 4. L'animazione deve funzionare senza errori 5. Verificate nei log che non ci siano messaggi di errore per file non trovati

---

C7 — male_black_shirt incompleto

File da modificare: `data/characters.json` e/o `scripts/rooms/character_controller.gd`

La soluzione consigliata (Opzione A) e' rimuovere temporaneamente il personaggio incompleto:

1. Aprite `data/characters.json`
2. Rimuovete l'intera sezione dedicata a `male_black_shirt`
3. Aprite `scripts/constants.gd` (o il file dove sono definiti i personaggi disponibili)
4. Rimuovete `male_black_shirt` dall'elenco dei personaggi selezionabili

Se preferite mantenere il personaggio con un fallback (Opzione C), aggiungete questa verifica in `character_controller.gd`:

```
# Prima di riprodurre qualsiasi animazione, verifichiamo che esista
# nelle SpriteFrames del personaggio corrente
func _play_animation(anim_name: String) -> void:
    # Null check: il nodo AnimatedSprite2D esiste?
    if _anim == null:
        push_error("CharacterController: AnimatedSprite2D non trovato")
        return

    # L'animazione esiste per questo personaggio?
    if _anim.sprite_frames.has_animation(anim_name):
        _anim.play(anim_name)  # si, riproduciamola
    else:
        # No, usiamo il fallback sicuro
        # idle_down e' l'unica animazione garantita per tutti i personaggi
        push_warning("Animazione '%s' non trovata, uso fallback" % anim_name)
        _anim.play("idle_down")
```

Come verificare: - **Opzione A**: Il personaggio `male_black_shirt` non appare piu' nella lista di selezione - **Opzione C**: Selezionate `male_black_shirt`, provate a camminare. Invece di crashare, il personaggio mostra l'animazione `idle_down`

A1 — Template _exit_tree() per tutti gli script

Obiettivo: Aggiungere la funzione di pulizia a 12 script.

Per ogni script, il procedimento e':

Passo 1: Aprite il file e trovate la funzione `_ready()`. Elencate tutte le righe che contengono `.connect(`:

```
# Esempio da room_base.gd _ready()
func _ready() -> void:
    SignalBus.room_changed.connect(_on_room_changed)      # segnale 1
    SignalBus.decoration_placed.connect(_on_decoration_placed)  # segnale 2
    SignalBus.decoration_removed.connect(_on_decoration_removed)  # segnale 3
```

Passo 2: Per OGNI `.connect` trovato, aggiungete una `.disconnect` corrispondente in `_exit_tree()`:

```
# Aggiungere alla fine del file
func _exit_tree() -> void:
    # Disconnettiamo OGNI segnale connesso in _ready()
    # Il check is_connected previene errori se gia' disconnesso
    if SignalBus.room_changed.is_connected(_on_room_changed):
        SignalBus.room_changed.disconnect(_on_room_changed)
    if SignalBus.decoration_placed.is_connected(_on_decoration_placed):
        SignalBus.decoration_placed.disconnect(_on_decoration_placed)
    if SignalBus.decoration_removed.is_connected(_on_decoration_removed):
        SignalBus.decoration_removed.disconnect(_on_decoration_removed)
```

Passo 3: Controllate se ci sono timer o tween creati nel file. Se si', aggiungeteli alla pulizia:

```
    # Fermare timer attivi
    if _timer and not _timer.is_stopped():
        _timer.stop()

    # Killare tween attivi
    if _tween and _tween.is_running():
        _tween.kill()
```

Elenco completo degli script da correggere, con i segnali da disconnettere (verificate aprendo ogni file):

1. **panel_manager.gd**: input handler
2. **shop_panel.gd**: segnali bottoni categorie e acquisto
3. **deco_panel.gd**: segnali bottoni (stub vuoto da completare)
4. **settings_panel.gd**: 4 segnali slider + 1 segnale option (stub vuoto da completare)
5. **room_base.gd**: 3 segnali SignalBus
6. **decoration_system.gd**: input handler
7. **room_grid.gd**: segnale `decoration_mode_changed`
8. **main_menu.gd**: tween + eventuali pannelli aperti
9. **menu_character.gd**: timer frame
10. **main.gd**: segnale `room_changed`
11. **music_panel.gd**: completare i restanti ~8 segnali non disconnessi
12. **character_controller.gd**: nessun segnale ma aggiungere null check su `_anim`

Come verificare: 1. Aprite il Profiler di Godot (menu Debug -> Profiler) 2. Osservate il conteggio degli oggetti in memoria 3. Cambiate scena (menu -> gioco -> menu) 20 volte 4. Il conteggio degli oggetti deve tornare al valore iniziale dopo ogni cambio 5. Se cresce costantemente, c'e' ancora un leak da trovare

---

A2 — FileDialog memory leak music_panel.gd

Le istruzioni dettagliate per questa correzione sono gia' state fornite nella Fase 2, Sezione 2.2 del Piano di Stabilizzazione. Il principio chiave e': creare il FileDialog una sola volta come variabile membro della classe, riutilizzarlo ad ogni click, e distruggerlo in `_exit_tree()`.

Come verificare: 1. Aprite il pannello musica 2. Cliccate "Importa" 20 volte (chiudendo il FileDialog ogni volta) 3. Controllate il Profiler: deve esserci UN SOLO FileDialog in memoria, non 20

---

13. Verifica e Testing

Come Verificare Ogni Fix

Questa tabella riassume come verificare che ogni correzione funzioni. Per ogni fix, e' descritto il test manuale da eseguire.

Fix	Come Verificare
C1 (inventario SQLite)	Salvare il gioco -> aprire il database SQLite con DB Browser -> eseguire `SELECT * FROM inventario` -> gli oggetti devono essere presenti
C2 (backup check)	Simulare disco pieno (o directory con permessi sola-lettura) -> salvare -> controllare i log per il messaggio di errore -> il salvataggio principale deve comunque funzionare
C3 (characters PK)	Creare 2 personaggi per lo stesso account -> eseguire `SELECT * FROM characters WHERE account_id = 1` -> devono esserci 2 righe
C4 (inventory schema)	Acquistare un oggetto -> verificare che `coins` nella tabella `accounts` sia diminuito e che l'oggetto sia in `inventario`
C5 (array mismatch)	Rinominare un file layer dello sfondo -> avviare il gioco -> non deve crashare
C6 (sprite typo)	Selezionare `male_old` -> farlo camminare verso il basso/lato -> l'animazione deve funzionare
C7 (black shirt)	Selezionare `male_black_shirt` (se ancora presente) -> muoversi in tutte le direzioni -> nessun crash
A1 (_exit_tree)	Aprire/chiudere un pannello 100 volte -> controllare il Profiler -> nessuna crescita costante di memoria
A2 (FileDialog)	Cliccare "Importa" 10 volte -> controllare il Profiler -> un solo FileDialog in memoria
A3 (race condition)	Cambiare personaggio rapidamente 20 volte -> nessun crash, nessun personaggio duplicato

Come Usare il Profiler di Godot

Il **Profiler** e' uno strumento integrato in Godot che vi permette di monitorare le prestazioni del gioco in tempo reale. E' fondamentale per verificare l'assenza di memory leak.

Come accedervi: 1. Avviate il gioco dall'editor di Godot (premete F5) 2. In basso, cliccate sulla tab "Debugger" 3. Selezionate "Profiler" dalla barra laterale 4. Abilitate il profiling cliccando "Start"

Cosa cercare per i memory leak: - Osservate il monitor "Object Count" (conteggio oggetti) - Eseguite l'azione sospetta (aprire/chiudere un pannello, cambiare stanza, ecc.) diverse volte - Se il conteggio cresce costantemente senza mai scendere, c'e' un memory leak - Se il conteggio sale e scende (ritorna a valori simili), la pulizia funziona

Come controllare i log: - I log vengono salvati in `user://logs/` (la directory varia per sistema operativo) - Su Windows: `%APPDATA%/Godot/app_userdata/MiniCozyRoom/logs/` - Su Linux: `~/.local/share/godot/app_userdata/MiniCozyRoom/logs/` - Su macOS: `~/Library/Application Support/Godot/app_userdata/MiniCozyRoom/logs/`

Nuovi Test da Scrivere

Oltre ai test gia' descritti nella Fase 5, ecco l'elenco completo dei file di test necessari:

```
tests/unit/
    test_audio_manager.gd      — tracce bounds, volume dB, modalita' playlist
    test_local_database.gd     — operazioni CRUD, schema, foreign keys
    test_room_base.gd          — spawn decorazioni, swap personaggio
    test_panel_manager.gd      — open/close lifecycle, mutua esclusione
    test_game_manager.gd       — caricamento cataloghi, cambiamenti di stato
    test_window_background.gd  — caricamento layer, allineamento array
```

Ordine di Esecuzione dei Test

Per una verifica completa, eseguite i test in questo ordine:

1. **Lint**: `gdlint v1/scripts/` — Verifica lo stile del codice (naming, struttura)
2. **Format**: `gdformat --check v1/scripts/` — Verifica la formattazione (indentazione, spazi)
3. **Test esistenti**: Eseguite i 5 file di test esistenti (48 test totali)
4. **Nuovi test**: Eseguite i 6 nuovi file di test
5. **Test manuale**: Giocate una sessione completa: menu -> stanza -> piazza decorazioni -> cambia personaggio -> ascolta musica -> salva -> chiudi -> riapri -> verifica che tutto sia stato salvato correttamente

---

14. Riferimenti e Risorse

Skill Files per Area di Correzione

Ogni area del progetto ha un file di riferimento (skill file) con best practice e pattern consigliati:

Area di Correzione	Skill File	Sezioni Chiave da Consultare
`_exit_tree()`, ciclo di vita nodi	`godot-engine-core.md`	"Node Lifecycle", "queue_free vs free"
Segnali, disconnessione	`godot-engine-core.md` + `godot-design-patterns.md`	"Signals" + "Observer Pattern"
Type hints, gestione errori	`gdscript-mastery.md`	"Type System", "Error Handling"
Collision, CharacterBody2D	`godot-2d-systems.md`	"CharacterBody2D", "StaticBody2D"
Ciclo di vita pannelli, drag-and-drop	`godot-ui-ux.md`	"Panel Lifecycle", "Drag-and-Drop"
Filtri texture, scaling	`pixel-art-rendering.md`	"Texture Filtering", "Integer Scaling"
Crossfade, volume, ambience	`godot-audio.md`	"Crossfade", "Volume Management"
Save/load, SQLite, migrazione	`godot-data-persistence.md`	"JSON Save", "SQLite", "Version Migration"
GdUnit4, asserzioni	`godot-testing.md`	"Assertion API", "Testing Autoloads"
FPS cap, caching, tween	`godot-performance.md`	"Dynamic FPS", "Tween Safety"
Export, HTML5, piattaforme	`godot-export-deploy.md`	"Platform Detection", "HTML5 Considerations"
Signal bus, dirty flag	`godot-design-patterns.md`	"Observer", "Dirty Flag", "Graceful Degradation"
Animazioni tween, walk-in	`godot-animation.md`	"Tween-Based Animation", "Walk-In Cinematic"

Documentazione Ufficiale Godot

- **Documentazione Godot 4**: https://docs.godotengine.org/en/stable/
- **Riferimento API GDScript**: https://docs.godotengine.org/en/stable/classes/index.html
- **Tutorial Segnali**: https://docs.godotengine.org/en/stable/getting_started/step_by_step/signals.html
- **Guida al Salvataggio**: https://docs.godotengine.org/en/stable/tutorials/io/saving_games.html
- **Guida all'Export**: https://docs.godotengine.org/en/stable/tutorials/export/index.html

Percorso di Apprendimento Consigliato

Se siete nuovi a Godot e volete approfondire i concetti trattati in questo audit, ecco un percorso di studio suggerito:

1. **Base**: Tutorial "Your First 2D Game" sulla documentazione ufficiale di Godot
2. **Segnali**: Capitolo "Signals" della documentazione ufficiale
3. **Scene e Nodi**: Capitolo "Scenes and Nodes" della documentazione ufficiale
4. **GDScript**: Riferimento completo del linguaggio sulla documentazione ufficiale
5. **Testing**: Documentazione di GdUnit4 su GitHub
6. **Database**: Tutorial SQLite per principianti (qualsiasi risorsa online)
7. **Pattern di Design**: Capitoli su Observer e Singleton nei libri di design patterns

---

15. Riepilogo Statistico

Categoria	Conteggio
File analizzati	48 (26 script + 9 scene + 5 dati + 5 test + 3 CI)
Righe di codice analizzate	~3500 (solo script GDScript)
Problemi CRITICI	7
Problemi ALTI	18
Violazioni architetturali	11
Problemi MEDI	30+
Problemi BASSI	8
Copertura test attuale	~15-20%
Copertura test target	50%+
Fasi di stabilizzazione	5
Nuovi file test necessari	6

Documento redatto come parte dell'audit pre-rilascio del progetto Mini Cozy Room. Autore: Renan Augusto Macena